

BASELINE · SECURITY REVIEW

Baseline

Mercury Round 3.

A security review of Mercury Round 3.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Baseline	Mercury Round 3	Guardian	January 21 – April 30, 2026	May 7, 2026

§ 00

Contents.

176 pages — overview, confidence ranking, scope, 109 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 57 findings	P. 14
	Remediation Review 1 · 26 findings	P. 87
	Remediation Review 2 · 12 findings	P. 127
	Remediation Review 3 · 7 findings	P. 148
	Remediation Review 4 · 3 findings	P. 161
	Remediation Review 5 · 4 findings	P. 167
§ 05	Disclaimer	P. 175
§ 06	About Guardian	P. 176

§ 01

Overview.

Guardian reviewed Mercury Round 3 across 6 review rounds. The final report includes 4 Critical, 9 High, 28 Medium, 33 Low, 35 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	0xBaseline/mercury
CLIENT	METHODOLOGY
Baseline	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2 · Remediation Review 3 · Remediation Review 4 · Remediation Review 5
PROJECT	MAIN REVIEW COMMIT
Mercury Round 3	513ed6e354c6
PREPARED BY	REMEDATION REVIEW 1 COMMIT
Guardian	9ad55b9b8430
AUDIT WINDOW	REMEDATION REVIEW 2 COMMIT
January 21 – April 30, 2026	6b80de4299c5
FINAL REPORT	REMEDATION REVIEW 3 COMMIT
May 7, 2026	2130c0f784c5

1.1 Confidence Ranking

1 / 5

Very Low Confidence

Guardian assigns a Confidence Ranking of 1 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	4	0	4	0	0
High	9	0	9	0	0
Medium	28	0	28	0	0
Low	33	0	33	0	0
Informational	35	0	35	0	0
All severities	109	0	109	0	0

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence ASSIGNED	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Mercury Round 3 in `0xBaSeLine/mercury`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Reserve tokens can be drained	Logical Error	CRITICAL	ACKNOWLEDGED	p. 14
C-02	Protocol reserves can be drained	Validation	CRITICAL	ACKNOWLEDGED	p. 16
H-01	Pool fees are lost when claimed	Logical Error	HIGH	ACKNOWLEDGED	p. 17
H-02	Vault withdrawals DOS due to solvency check	DoS	HIGH	ACKNOWLEDGED	p. 18
H-03	Stale K enables reserve extraction	Logical Error	HIGH	ACKNOWLEDGED	p. 20
M-01	Configured feeRecipient ignored in payouts	Logical Error	MEDIUM	ACKNOWLEDGED	p. 23
M-02	Staking rewards can be globally grieved & frozen	DoS	MEDIUM	ACKNOWLEDGED	p. 24
M-03	Reverting swaps due to underflow	DoS	MEDIUM	ACKNOWLEDGED	p. 26
M-04	User favorable rounding in sell path	Rounding	MEDIUM	ACKNOWLEDGED	p. 27
M-05	BLV translation uses outdated state	Logical Error	MEDIUM	ACKNOWLEDGED	p. 28
M-06	Vault exit loss not propagated to curve math	Math	MEDIUM	ACKNOWLEDGED	p. 29
M-07	Zero share deposits can brick protocol sweeps	DoS	MEDIUM	ACKNOWLEDGED	p. 31
M-08	takeReserves() can result in 0 shares minted	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 33

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-09	BCredit.defaultSelf off curve	Logical Error	MEDIUM	ACKNOWLEDGED	p. 34
M-10	Swap fee can be bypassed when selling all tokens	Gaming	MEDIUM	ACKNOWLEDGED	p. 36
M-11	Entering vault for a bToken bricks its pool	DoS	MEDIUM	ACKNOWLEDGED	p. 37
M-12	Convexity parameter is being overrelaxed	Math	MEDIUM	ACKNOWLEDGED	p. 38
M-13	Underreported supply drains shared reserves	Gaming	MEDIUM	ACKNOWLEDGED	p. 40
L-01	Swaps bricks when convexityExp > 2e18	DoS	LOW	ACKNOWLEDGED	p. 41
L-02	Unaccounted dust desync reserves accounting	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 42
L-03	Pool creation may fail due to underflow	DoS	LOW	ACKNOWLEDGED	p. 43
L-04	Unused code in quote functions	Superfluous Code	LOW	ACKNOWLEDGED	p. 45
L-05	Smaller swaps reverting due to fee rounding to 0	DoS	LOW	ACKNOWLEDGED	p. 46
L-06	Remove final correction from buy/sell paths	Best Practices	LOW	ACKNOWLEDGED	p. 48
L-07	Asymmetrical validation for buy path	Validation	LOW	ACKNOWLEDGED	p. 50
L-08	Incorrect event emission for hook swaps	Events	LOW	ACKNOWLEDGED	p. 51
L-09	solveBuy() can revert due to overflow	Math	LOW	ACKNOWLEDGED	p. 52
L-10	Buy fee path underflow can revert swaps	DoS	LOW	ACKNOWLEDGED	p. 53
L-11	Buffer can grow during sells	Math	LOW	ACKNOWLEDGED	p. 54
L-12	Hook sweep can revert batched swaps	DoS	LOW	ACKNOWLEDGED	p. 55

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-13	Recorded reserves drift due to vault operations	Math	LOW	ACKNOWLEDGED	p. 56
L-14	Hook sweep can under-credit vault	Math	LOW	ACKNOWLEDGED	p. 58
L-15	Harvest buffer not retained on vault exit	Math	LOW	ACKNOWLEDGED	p. 60
L-16	Harvest buffer can be dynamically adjusted	Suggestion	LOW	ACKNOWLEDGED	p. 61
L-17	Pool reserves overstated on createPool()	Rounding	LOW	ACKNOWLEDGED	p. 62
L-18	Duplicate claim leaves enable grieving	Trust Assumptions	LOW	ACKNOWLEDGED	p. 63
L-19	Leverage fee charged on max borrow	Math	LOW	ACKNOWLEDGED	p. 64
L-20	Ratio rounding favors users on buys and sells	Rounding	LOW	ACKNOWLEDGED	p. 65
I-01	Missing guard causes sell side reverts	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 66
I-02	Excess repayment funds not refunded	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 67
I-03	Repay preview revert when debt is 0	Suggestion	INFORMATIONAL	ACKNOWLEDGED	p. 68
I-04	harvestYield has a return but doesn't return it	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 69
I-05	Missing upper bound blocks pool creation	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 70
I-06	isExecutor() can use dirty address	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 72
I-07	Relay routes are unnecessary	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 73
I-08	Incorrect ceil comments	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 75

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-09	Redundant mulWad() in active price computation	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 76
I-10	Unreachable edge cases in computeSwap()	Suggestion	INFORMATIONAL	ACKNOWLEDGED	p. 77
I-11	Binary search loop can break earlier	Gas Optimization	INFORMATIONAL	ACKNOWLEDGED	p. 78
I-12	Dead Pool.shareBalance state field	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 79
I-13	K decreases due to computation inconsistencies	Rounding	INFORMATIONAL	ACKNOWLEDGED	p. 80
I-14	Vaults are a security risk	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 81
I-15	Unused previewTakeReserves() helper	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 82
I-16	Outdated refund ordering comment	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 83
I-17	Invariant error lacks drop context	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 84
I-18	Check computed blv against book price	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 85
I-19	Lack of BToken parameters validation	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 86

REMEDIATION REVIEW 1

H-01	K drops due to overcharged fee	Rounding	HIGH	ACKNOWLEDGED	p. 87
H-02	Sell side buffer rounding breaks swap	DoS	HIGH	ACKNOWLEDGED	p. 89
H-03	Inverse pow rounding inflates buy side fees	Rounding	HIGH	ACKNOWLEDGED	p. 92
H-04	Convexity reset removes buy side price impact	Math	HIGH	ACKNOWLEDGED	p. 95

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-05	Round trip swaps enable reserves drain	Math	HIGH	ACKNOWLEDGED	p. 97
M-01	computeSwap can DoS swaps	DoS	MEDIUM	ACKNOWLEDGED	p. 99
M-02	computeNextBLV can DoS swaps	DoS	MEDIUM	ACKNOWLEDGED	p. 101
M-03	Yield distribution can be violated	Rewards	MEDIUM	ACKNOWLEDGED	p. 103
M-04	Reopen buy bypasses stored invariant	Logical Error	MEDIUM	ACKNOWLEDGED	p. 105
M-05	Non additive IL fee exploitable	Logical Error	MEDIUM	ACKNOWLEDGED	p. 107
M-06	TPS decay computed but never applied	Logical Error	MEDIUM	ACKNOWLEDGED	p. 109
L-01	Pool initialization may fail	Validation	LOW	ACKNOWLEDGED	p. 111
L-02	Impossible to default if resulting <code>circ == 0</code>	DoS	LOW	ACKNOWLEDGED	p. 112
L-03	Hook swap bypasses reserve dust accounting	Logical Error	LOW	ACKNOWLEDGED	p. 113
L-04	Hidden reserves in hook swaps	Logical Error	LOW	ACKNOWLEDGED	p. 115
I-01	Initial credits may be unclaimable	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 116
I-02	Pool creation cannot be paid with native tokens	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 117
I-03	Outdated comment	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 118
I-04	Lack of validation for <code>creatorFeePct</code>	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 119
I-05	Redundant function parameter	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 120
I-06	<code>previewRepay()</code> should return <code>debtToRepay</code>	Suggestion	INFORMATIONAL	ACKNOWLEDGED	p. 121
I-07	Redundant <code>mulWad()</code> in active price computation	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 122

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-08	Unused errors	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 123
I-09	Comment mismatch on premium threshold	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 124
I-10	Confusing plural used for variable naming	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 125
I-11	K can overflow	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 126

REMEDIATION REVIEW 2

C-01	Permanent BLV breakage	Logical Error	CRITICAL	ACKNOWLEDGED	p. 127
C-02	Pending commit can be permanently poisoned	Logical Error	CRITICAL	ACKNOWLEDGED	p. 129
H-01	Active price mismatch censors exact buys	DoS	HIGH	ACKNOWLEDGED	p. 131
M-01	Stale invariant doesn't account for fees	Logical Error	MEDIUM	ACKNOWLEDGED	p. 133
M-02	Mixed direction block pricing not correct	Logical Error	MEDIUM	ACKNOWLEDGED	p. 135
M-03	Swaps revert due to fee subtraction underflow	DoS	MEDIUM	ACKNOWLEDGED	p. 137
M-04	Lower domain failure in <code>_tryRecomputeMaxReserves</code>	DoS	MEDIUM	ACKNOWLEDGED	p. 139
M-05	Sell sign inversion from invariant rounding	DoS	MEDIUM	ACKNOWLEDGED	p. 141
L-01	Dripping asymmetry when pending yield is 0	Gaming	LOW	ACKNOWLEDGED	p. 143
L-02	BLens may return stale curve parameters	Logical Error	LOW	ACKNOWLEDGED	p. 145
I-01	<code>BCredit.claim()</code> integration notes	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 146
I-02	Not all tokens can be bought	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 147

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
REMEDIATION REVIEW 3					
M-01	Reallocated fees commit divergency	DoS	MEDIUM	ACKNOWLEDGED	p. 148
L-01	Undistributed fees when circ reaches zero	Logical Error	LOW	ACKNOWLEDGED	p. 150
L-02	Buffer threshold precision mismatch	Math	LOW	ACKNOWLEDGED	p. 152
L-03	Fee rounding favors buyer over protocol	Rounding	LOW	ACKNOWLEDGED	p. 154
L-04	Stale BLV in BCredit and quoteLeverage	Logical Error	LOW	ACKNOWLEDGED	p. 156
L-05	computeNextBLV() overflow DOSes pool	DoS	LOW	ACKNOWLEDGED	p. 158
I-01	Integration note completeness	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 160
REMEDIATION REVIEW 4					
L-01	Safety threshold precision mismatch	Rounding	LOW	ACKNOWLEDGED	p. 161
L-02	View reserves inflated by unstripped surplus	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 163
I-01	Quote slippage is cumulative, not per-trade	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 166
REMEDIATION REVIEW 5					
M-01	Missed removal leave hidden extractable reserves	Logical Error	MEDIUM	ACKNOWLEDGED	p. 167
M-02	Snapshot sell miscalculate circulation state	Logical Error	MEDIUM	ACKNOWLEDGED	p. 169
M-03	Same-block terminal exit pays below BLV floor	Logical Error	MEDIUM	ACKNOWLEDGED	p. 171
I-01	Rebalance unlocks surplus from initial credit	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 173

CRITICAL **C-01**

ACKNOWLEDGED

Reserve tokens can be drained

CATEGORY Logical Error
LOCATION BFactory.sol
REVIEW Main Review

DESCRIPTION

```
In BFactory.createPool()  
  
// pool asset accounting  
pool.totalReserves = (params.initialPoolReserves + params.initialDebt).toUint128();  
// only transfer initial reserves from caller.  
pool.reserve.takeReserves(msg.sender, params.initialPoolReserves);
```

we only collect `initialPoolReserves` from the creator.

and credit the pool with `initialDebt` as if it were reserves by adding it into `pool.totalReserves`.

a permissionless pool creator can set `initialDebt` arbitrarily large and make the AMM believe the pool has huge reserves

The Sell path uses curve params derived from `totalReserves`, the curve now believes the pool is backed by a huge amount of reserves

```
params_.reserves = NormalizeLib.normalizeWad(pool.totalReserves, pool.reserveDecimals);
```

And every place that pays out reserves uses the ERC20 balance held by the Relay, not per pool separated balance

```
reserve.giveReserves(msg.sender, _deltaUserReserves.toUint256(), false)
```

`VaultLib.giveReserves()` pulls from

```
State.Allocation storage allocation = State.meta().allocations[_reserve];
```

An attacker can drain reserve tokens of other pools by selling, as demonstrated in the PoC

<https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/master/test/unit/InitialDebtReserveDrain.poc.t.sol>

C-01 Reserve tokens can be drained

CONTINUED

— RECOMMENDATION

If initialDebt needs to be reflected in accounting, consider tracking it separately

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/e81f3a58403bde43d7b37b9a77bdc3d851ae8f47/src/components/BFactory.sol#L228>

CRITICAL C-02

ACKNOWLEDGED

Protocol reserves can be drained

CATEGORY Validation
LOCATION .
REVIEW Main Review

DESCRIPTION

Related with the added solvency check for C-01

```
(uint256 maxBorrow,uint256 fee) = BCredit(address(this)).getBorrowForCollateral();  
require(maxBorrow + fee >= params.initialDebt, InsolventInitialCreditPosition());
```

getBorrowForCollateral() depends on blvPrice, and blvPrice depends on pool.totalReserves, and we already added `initialDebt` into pool.totalReserves

by inflating `initialDebt`, we inflate pool.totalReserves then inflate bookPrice / BLV, which inflate maxBorrow + fee

So attacker could still drain pool reserves of any token

<https://gist.github.com/GuardianAudits/8c790f2784f855ad91ff1ef85688885b>

RECOMMENDATION

consider removing initialDebt for permissionless pool creation

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/7d128420821b922870e5a844083b309cd96ec0c6/src/libraries/GuardLib.sol#L89>

HIGH **H-01**

ACKNOWLEDGED

Pool fees are lost when claimed

CATEGORY Logical Error
LOCATION BFactory.sol
REVIEW Main Review

— DESCRIPTION

In `createPool` the `params.creator` and `params.feeRecipient` values are validated but never persisted to pool state

we never do `pool.creator = params.creator;`

and `pool.feeRecipient = params.feeRecipient;`

so after pool creation, `State.pool(params.bToken).creator` and `State.pool(params.bToken).feeRecipient` stays at its default value `address(0)`

And because `pool.creator` is never set, the intended creator can't transfer creator later with `transferCreator()`

In `_claimPoolFees`

```
pool.reserve.giveReserves(pool.creator, creatorFees, false);  
pool.reserve.giveReserves(meta.protocolFeeRecipient, protocolFees, false);
```

because both values are 0, any fees collected will be irreversibly lost when claiming it

— RECOMMENDATION

Inside `createPool`, store the creator and the fee recipient

```
pool.creator = params.creator;  
pool.feeRecipient = params.feeRecipient;
```

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/e81f3a58403bde43d7b37b9a77bdc3d851ae8f47/src/components/BFactory.sol#L201>

HIGH **H-02**

ACKNOWLEDGED

Vault withdrawals DOS due to solvency check

CATEGORY DoS
LOCATION VaultLib.sol#L312-314
REVIEW Main Review

DESCRIPTION

`VaultLib.depositToVault` records `depositedReserves` using `previewRedeem(shares)`, so the tracked assets are rounded down and are equal to the amount of assets owned by the contract.

`VaultLib.withdrawFromVault()` then burns shares via `vault.withdraw(_reserveAmount,...)`, which internally uses `previewWithdraw` (rounds up). After this burn, `_ensureVaultSolvency()` checks `previewRedeem(remainingShares) >= depositedReserves`.

Because `previewWithdraw()` can burn extra shares for the same asset amount, the remaining share balance can preview to less than the tracked reserves even when the vault is solvent. This will result in unexpected DOS cases while normally using the protocol.

Example:

- totalAssets=2, totalShares=3.
- deposit 2 assets → receive 3 shares, depositedReserves=2.
- withdraw 1 asset → previewWithdraw(1)=2 shares burned.
- depositedReserves is decreased to 1
- remaining shares=1, previewRedeem(1)=0 while depositedReserves=1, so the check reverts despite no real loss.

RECOMMENDATION

Reconsider if this check is really needed. You can also switch to tracking shares and comparing them against the real share balance of the protocol.

PROOF OF CONCEPT https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/SolvencyDOS_POC/test/SolvencyDOS_POC.t.sol

RESOLUTION

Status: Acknowledged.

H-02 Vault withdrawals DOS due to solvency check

CONTINUED

<https://github.com/0xBaseline/mercury/blob/7352fe69811a35a143821ffedf7023961ea80c2a/src/libraries/VaultLib.sol#L136>

HIGH **H-03**

ACKNOWLEDGED

Stale K enables reserve extraction

CATEGORY Logical Error
LOCATION .
REVIEW Main Review

DESCRIPTION

updateAccounting() computes and adds liquidityFee_ into pool.totalReserves

recordSwap() decides whether to recompute maker.lastInvariant, the curve K using a different threshold test

here liquidity fee gating uses a ratio, wad division

```
// _updateAccounting
liquidityFee_ = _feesReceived - LIQUIDITY_GROWTH_FEE_SHARE.mulWad(_feesReceived);
if (uint256(pool.totalBTokens).divWad(pool.totalSupply) >= 0.95e18) {
liquidityFee_ = 0;
}
```

If the pool has $\geq 95\%$ of supply inside totalBTokens/totalSupply $\geq 95\%$, then no liquidityFee

If $< 95\%$, then liquidityFee is kept inside the pool and added into pool.totalReserves

but here, Invariant recompute gating uses a floored product

```
// _recordSwap
if (pool.totalBTokens < pool.totalSupply.mulWad(0.95e18)) {
newInvariant = CurveLib.computeInvariant(getCurveParams(_bToken));
}
maker.lastInvariant = newInvariant;
```

So invariant recompute happens if

```
totalBTokens < floor( totalSupply * 0.95 )
```

the vulnerability here is that we are using two different definitions of inside / outside the 95% safety band

Liquidity fee uses

```
floor(totalBTokens * 1e18 / totalSupply ) >= 0.95e18
```

H-03 Stale K enables reserve extraction

CONTINUED

Invariant refresh uses

```
totalBTokens < floor(totalSupply * 0.95)
```

Those differ at the boundary whenever $0.95 * \text{totalSupply}$ is not an integer, creating a fee kept but K not updated inconsistent state

<https://gist.github.com/GuardianAudits/83303e7af8afb67a41b0b84355200a69>

That leaves the system in an inconsistent state

`pool.totalReserves` has been increased by a fee that is intended to grow liquidity

but `maker.lastInvariant K` is still the pre fee value

Since `CurveLib.computeSwap()` treats `lastInvariant` as the source of truth

pricing is now computed using stale K against a reserves value that already includes extra buffer

after every swap ends exactly at $\text{totalBTokens} = \text{floor}(0.95 * \text{totalSupply})$, the pool reserves already include the liquidity fee, but K `lastInvariant` is still the old value,

so the next swap that moves `totalBTokens` one step below that floor refreshes `lastInvariant`, causing a sudden curve jump

An attacker can arbitrage this in one transaction

1 - Buy a tiny amount while K is stale (cheaper)

2 - That buy crosses the floor, so `_recordSwap` updates `lastInvariant`

3 - The update now locks in the previously added liquidity fee into the curve

4 - Immediately sell back the same amount against the updated curve (higher price), pocketing reserves

In the poc, the attacker buys 1900 bTokens for 90 reserves using stale K,

crossing the boundary refreshes K after the buy, attacker sells back for 1906 reserves, netting 1816 profit

<https://gist.github.com/GuardianAudits/1bbd127b943b837305385d214ddb6de6>

— RECOMMENDATION

make the liquidity fee decision and the invariant refresh decision use the same predicate

H-03 Stale K enables reserve extraction

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/77094087b38cb7ccd0a76dd572714a6fc31bade1/src/libraries/MakerLib.sol#L235>

MEDIUM **M-01**

ACKNOWLEDGED

Configured feeRecipient ignored in payouts

CATEGORY Logical Error
LOCATION BController.sol
REVIEW Main Review

DESCRIPTION

The configurable feeRecipient set via setFeeRecipient is never used when distributing creator fees. claimPoolFees/claimPoolFees send creator fees to pool.creator, making FeeRecipientSet ineffective. This misroutes funds compared to configuration intent and prevents creators from delegating revenue to a separate recipient without transferring creator ownership

RECOMMENDATION

We need to replace the payout target for creator fees with pool.feeRecipient if set, falling back to pool.creator only if feeRecipient is unset.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/e81f3a58403bde43d7b37b9a77bdc3d851ae8f47/src/components/BController.sol#L229>

MEDIUM **M-02**

ACKNOWLEDGED

Staking rewards can be globally grieved & frozen

CATEGORY DoS
LOCATION BStaking.sol
REVIEW Main Review

DESCRIPTION

This vulnerability allow an attacker to keep the global accumulator from ever increasing, which means `_getEarned()` for everyone returns the old `prevEarned` for very long time, while `pool.pendingYield` continues to grow. Then the attacker can stake right before distribution resumes and capture rewards that accrued while they were not staked or Just keep doing the DoS

`getAccumulator()` can decide not enough yield to distribute

```
uint256 yield = min(tokensPerSecond_ * timeElapsed, pendingYield);  
uint256 accumulatorIncrease = yield.fullMulDiv(RAY, totalStaked);  
// @audit, not enough yield to distribute evenly, return early  
if (accumulatorIncrease == 0) return (accumulator_, 0, tokensPerSecond_);
```

So if `yield * RAY < totalStaked`, no accumulator increase happens, and `newYield_ = 0`

`_sync()` still updates `lastUpdated` even when nothing was distributed

```
(accumulator, newYield, tokensPerSecond) = getAccumulator(_bToken);  
// update the last updated timestamp every time  
if (staking.lastUpdated != block.timestamp)  
    staking.lastUpdated = block.timestamp.toUint32();  
// @audit, return early if user's accumulator up to date  
if (accumulator == staking.accounts[_user].userAccumulator) return;
```

That means an attacker can keep `timeElapsed` tiny forever, because it's always `block.timestamp - lastUpdated`, ensuring `yield` stays tiny, ensuring `accumulatorIncrease` stays 0

So rewards never move from `pool.pendingYield` into the stakers accumulator, and users `earned` won't grow

An attacker only needs a public function that calls `_sync()`

M-02 Staking rewards can be globally grieved & frozen

CONTINUED

BStaking.deposit() is public and does not require `_amount > 0`

Calling `deposit(_bToken, attacker, 0)` is enough to run `_sync()`

So the attacker can cheaply send a Tx repeatedly to keep resetting `lastUpdated` and stakers cannot accrue `earned`

This can persist indefinitely as long as the attacker keeps calling.

— RECOMMENDATION

We need to prevent `lastUpdated` from being advanced when the accumulator didn't advance

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/3ab36e01c5dadf20ce472440c447fb867a279e48/src/components/BStaking.sol#L211>

MEDIUM **M-03**

ACKNOWLEDGED

Reverting swaps due to underflow

CATEGORY DoS
LOCATION CurveLib.sol#L171
REVIEW Main Review

DESCRIPTION

`CurveLib.computeFee()` computes the `IL` fee by subtracting `ΔB` from `|Δc| * marginalPremium`.

```
// IL Fee = difference between marginal price and integral price  
fee_ = absDelta.mulWadUp(marginalPremium) - (newBufferDown - buffer);
```

In the ideal case, the LHS of the equation should always be greater than the RHS. However, for some swaps LHS is actually lower, which results in swaps reverting due to underflow.

The same is true for the else branch as well:

```
fee_ = (buffer - _newBuffer) - (absDelta.mulWad(marginalPremium));
```

RECOMMENDATION

Consider using `zeroFloorSub()` when calculating the IL fee.

Buy:

```
fee_ = (absDelta.mulWadUp(marginalPremium)).zeroFloorSub(newBufferDown - buffer);
```

Sell:

```
fee_ = (buffer - _newBuffer).zeroFloorSub(absDelta.mulWad(marginalPremium));
```

In addition, you can add a maximum tolerable negative delta validation.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-fuzz-1768594995707/commit/b09b84c9fe85b1478252def87036217c86eab69a>

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/4735c459372aea7027f20c2f53019558e3f90ab7/src/libraries/CurveLib.sol#L152>

MEDIUM **M-04**

ACKNOWLEDGED

User favorable rounding in sell path

CATEGORY Rounding
LOCATION CurveLib.sol#L121
REVIEW Main Review

DESCRIPTION

In `CurveLib.computeSwap` the sell branch is explicitly documented as “protocol-favorable” rounding. It uses `fullMulDivUp` for `newBuffer`, which rounds up to make reserves larger and the user payout smaller. However, the BLV component is added with `mulWad`, which rounds down:

```
newReserves = newBuffer + BLV * c1
```

Using `mulWad` here slightly reduces `newReserves`, which makes the user receive more than the protocol-favorable rounding intent. This is inconsistent with the stated rounding policy for sells and can leak small amounts of value on each trade.

Over time, this can lead to swaps where the resulting `K` is smaller than the previous one even though `BLV` and `n` didn't change. Such swaps will revert with the `InvariantDecreased()` error.

RECOMMENDATION

Use `mulWadUp` for the BLV term in the sell branch so that `newReserves` is rounded up consistently:

```
- newReserves = newBuffer + BLV.mulWad(c1)
+ newReserves = newBuffer + BLV.mulWadUp(c1)
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-fuzz-1768594995707/blob/e76c21293bb6085139648f425a547a00cfa7d857/test/fuzzing/FoundryRound3.sol#L18-L343>

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/3ae71f63f48baa4fa50d0abcbabf55a7aaff801f/src/libraries/CurveLib.sol#L121>

MEDIUM **M-05**

ACKNOWLEDGED

BLV translation uses outdated state

CATEGORY Logical Error
LOCATION MakerLib
REVIEW Main Review

— DESCRIPTION

MakerLib.recordSwap stores a memory copy of State.Pool before mutating state and then uses that snapshot to decide whether to translate the BLV floor price

(checking if `pool.totalBTokens < pool.totalSupply * threshold`), Because this check happens after `updateAccounting()` but reads from the stale memory copy,

BLV translation decisions can be made using pre trade values, causing incorrect or delayed BLV updates and inconsistent curve evolution

swaps that cross the 95% boundary can wrongly skip or wrongly trigger BLV translation (one trade late/early)

— RECOMMENDATION

make the BLV translation threshold read post trade storage, not a pre trade memory snapshot

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/e21011f5a7214ea3182a1849523d9651b1f008db/src/libraries/MakerLib.sol#L175>

MEDIUM **M-06**

ACKNOWLEDGED

Vault exit loss not propagated to curve math

CATEGORY Math
LOCATION VaultLib
REVIEW Main Review

DESCRIPTION

`exitVaultUnsafe` realizes a loss but doesn't propagate it into pool / curve accounting,

`BController.exitVaultUnsafe(ERC20 _reserve)` is exposed as a Relay route and calls

`VaultLib.exitVaultUnsafe(ERC20 _reserve)`

```
function exitVaultUnsafe(ERC20 _reserve) external permissioned nonReentrant returns (
    uint256 amount_,
    uint256 loss_
) {
    (amount_, loss_) = _reserve.exitVaultUnsafe();
    emit VaultExited(address(_reserve), amount_, 0, loss_);
}
```

And in `VaultLib.exitVaultUnsafe`

```
redeemed_ = vault.redeem(vault.balanceOf(address(this)), address(this), address(this));
uint256 depositedReserves = allocation.depositedReserves;
// requires a loss
require(redeemed_ < depositedReserves, VaultLib_ExitBalanceMismatch());
loss_ = depositedReserves - redeemed_;
// wipe vault config and accounting
allocation.depositedReserves = 0;
allocation.idleReserves = 0;
allocation.vault = ERC4626(address(0));
```

`ExitVaultUnsafe` successfully exits an ERC4626 vault when there is a loss, `redeemed_ < depositedReserves`, and returns `loss_`, but it does not apply that loss anywhere else

In particular, it does not update

`State.pool(bToken).totalReserves`, the reserves number used by the AMM curve / pricing

`State.maker(bToken).lastInvariant`, the K source of truth used by `CurveLib.computeSwap`

M-06 Vault exit loss not propagated to curve math

CONTINUED

So after an unsafe exit

Actual reserves available to pay users are smaller because the vault lost funds

But Internal accounting still thinks the old amount exists

After `exitVaultUnsafe`, users can trade / withdraw based on stale, overstated `pool.totalReserves`. Early sellers can extract the remaining reserves at the pre loss price until the contract balance is exhausted, leaving late users unable to redeem

— RECOMMENDATION

If markets will not kept open after `exitVaultUnsafe`, then it would be safe

Otherwise apply the realized vault loss to the internal pool accounting and curve invariant for every pool that uses `_reserve`

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/717218cc7a92dd79b275ece952864a42f3193f87/src/components/BController.sol#L113>

MEDIUM **M-07**

ACKNOWLEDGED

Zero share deposits can brick protocol sweeps

CATEGORY DoS
LOCATION SweepLib
REVIEW Main Review

DESCRIPTION

Inside `VaultLib.depositIdleReserves()`

```
function depositIdleReserves(ERC20 _reserve) internal {
    State.Allocation storage allocation = State.meta().allocations[_reserve];
    ERC4626 vault = allocation.vault;
    uint256 idleReserves = allocation.idleReserves;
    if (!isAllocated(vault) || idleReserves == 0) return;
    uint256 harvestableYield = getHarvestableYield(allocation);
    if (harvestableYield == 0) return;
    uint256 vaultAssetValue = convertDepositToAssets(_reserve, idleReserves);
    uint256 precisionError = idleReserves.zeroFloorSub(vaultAssetValue);
    if (harvestableYield > precisionError) {
        allocation.idleReserves = 0;
        depositToVault(allocation, _reserve, idleReserves); // @audit, may revert / may mint 0 shares
    }
}
```

And `depositIdleReserves()` which is reachable from user flows via `SweepLib.executeSweep()`

```
if (VaultLib.isAllocated(allocation.vault)) {
    if (VaultLib.convertDepositToAssets(reserve, reserveBalance) > 0) {
        VaultLib.depositToVault(allocation, reserve, reserveBalance);
    } else {
        allocation.idleReserves += reserveBalance.toUint128();
        VaultLib.depositIdleReserves(reserve); // @audit this is the dangerous path
    }
}
```

M-07 Zero share deposits can brick protocol sweeps

CONTINUED

a normal user triggered action that causes a sweep can end up calling `depositIdleReserves()`, which is gated by `getHarvestableYield()`

`depositIdleReserves()` can attempt to deposit `idleReserves` even when `convertDepositToAssets(_reserve, idleReserves) == 0`

That case means

`vault.previewDeposit(idleReserves)` is effectively 0 shares or redeem value 0

Yet if `getHarvestableYield(allocation) > idleReserves` because there is enough yield, then

`precisionError = idleReserves - 0 = idleReserves`

condition `harvestableYield > precisionError` becomes `harvestableYield > idleReserves`

deposit is attempted anyway, It could cause a global DoS, common with [Solmate ERC4626] (<https://github.com/transmissions11/solmate/blob/89365b880c4f3c786bdd453d4b8e8fe410344a69/src/tokens/ERC4626.sol#L48>)

Many ERC4626 vaults revert on `deposit()` if `shares == 0`

So if `idleReserves` is still too small to mint shares

`depositToVault()` calls `vault.deposit(idleReserves, address(this))`, Vault reverts, the entire sweep reverts

and because sweeps are invoked in many important paths (swaps, staking syncs, fee claims) this can brick core interactions

This DoS is triggerable because users can create tiny `idleReserves` via very small swaps or any path that causes `reserveBalance` in the pool manager to be too small to deposit, and later force sweeps again.

— RECOMMENDATION

We must not attempt to deposit idle reserves unless the vault will mint non zero shares

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/master/src/libraries/SweepLib.sol#L107>

MEDIUM **M-08**

ACKNOWLEDGED

takeReserves () can result in 0 shares minted

CATEGORY Unexpected Behavior
LOCATION VaultLib.sol#L76-78
REVIEW Main Review

DESCRIPTION

`SweepLib.executeSweep()` checks the assets that were just swept will result in non-zero assets if deposited to the vault. If not, the idle reserves are increased instead.

`VaultLib.convertDepositToAssets(reserve, reserveBalance) > 0`

However, this logic is absent in `takeReserves()`. There `depositToVault()` is called with the exact amount unconditionally. If this amount is small enough to cause 0 shares minting, the result is either:

- loss of assets, since the relay receives 0 shares
- DOS for the whole transaction if the vault implementation reverts on 0 shares minted, for example

[Solmate]

(<https://github.com/transmissions11/solmate/blob/89365b880c4f3c786bdd453d4b8e8fe410344a69/src/tokens/ERC4626.sol#L48>).

Two very common paths enabling this issue are `BSwap.buyTokensExactIn()` and `BSwap.sellTokensExactOut()` because they use an estimation amount and the difference between it and the actual user amount is stored in `dustFee`, which is later passed as an argument to `takeReserves()`.

RECOMMENDATION

Add the same check to `takeReserves()` - if the current amount will mint 0 value, add it to idle reserves and call `depositIdleReserves()`.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/94a9b5d2c3c6e18229f0b4d65d19ec9159743d96/src/libraries/VaultLib.sol#L116>

MEDIUM **M-09**

ACKNOWLEDGED

BCredit.defaultSelf off curve

CATEGORY Logical Error
LOCATION BCredit
REVIEW Main Review

DESCRIPTION

In `BCredit.defaultSelf()`

```
pool.totalBTokens += collateral.toUint128();  
pool.totalReserves -= debt.toUint128();
```

This function directly mutates the AMM core state variables `totalBTokens`, `totalReserves` that drive pricing inside `MakerLib`, `CurveLib`

But it does not update the curve source of truth invariant

`State.maker(_bToken).lastInvariant`

That invariant is what `CurveLib.computeSwap()` uses to calculate swap deltas.

The AMM uses `maker.lastInvariant` as K

Swaps don't recompute K from the current reserves / supply, they use

`maker.lastInvariant`, stored K

plus current `pool.totalReserves` / `pool.totalBTokens` / `circ`

So `lastInvariant` must remain consistent with `pool.totalReserves`, `pool.totalBTokens`, `circ`, `BLV`, `convexityExp``

`defaultSelf()` changes `pool.totalBTokens`, `pool.totalReserves`

But leaves `maker.lastInvariant` unchanged.

That means after a default:

`pool.totalReserves` / `pool.totalBTokens` reflect a new curve point

But `maker.lastInvariant` still represents the old curve surface

So the pool is now off curve, and the next swap will compute prices from a stale K

M-09 BCredit.defaultSelf off curve

CONTINUED

RECOMMENDATION

After mutating `pool.totalBTokens` / `pool.totalReserves` in `defaultSelf()`, recompute and store the invariant

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/efff95228c9d02d54292a8a0c4b5a34bb022b79a/src/components/BCredit.sol#L263>

MEDIUM **M-10**

ACKNOWLEDGED

Swap fee can be bypassed when selling all tokens

CATEGORY Gaming
LOCATION CurveLib.sol#L187
REVIEW Main Review

DESCRIPTION

When all tokens are being sold to the pool, a flat `swapFee` is applied on top of the `BLV` price and the user receives reserves below `BLV` per `bToken`.

```
if (c1 == 0) {  
    uint256 blvValue = _params.BLV.mulWad(_params.circ);  
    uint256 receipt = blvValue.mulWad(WAD - _params.swapFee);  
    return (int256(receipt), _params.reserves - receipt);  
}
```

However, during normal sells `_computeFee()` applies the fee only to the `marginalPremium`, which depends on the buffer.

```
fee_ += absDelta.mulWadUp(marginalPremium.mulWadUp(_p.swapFee));
```

In a situation where a user will sell all the `bTokens` to the pool, they can split the swap in two parts: large and small one. For the big swap, they will end up paying `swapFee` only on the new `marginalPremium` which is small because the new buffer is small as well. Then they will pay `swapFee` for `BLV` only for the small swap.

RECOMMENDATION

Consider applying the `swapFee` to `BLV + marginalPremium` in `computeFee()`

```
- fee_ += absDelta.mulWadUp(marginalPremium.mulWadUp(_p.swapFee));  
+ fee_ += absDelta.mulWadUp((_p.BLV + marginalPremium).mulWadUp(_p.swapFee));
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1768594959614/commit/67d0daae1cba74ebda8edc52b63780f0a0e878b0>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-11**

ACKNOWLEDGED

Entering vault for a bToken bricks its pool

CATEGORY DoS
LOCATION `src/components/BController.sol#L100`
REVIEW Main Review

DESCRIPTION

When a bToken is also used as a reserve token, calling `setVault()` on that reserve deposits the entire bToken balance held by the protocol into the vault (enterVault uses `_reserve.balanceOf(address(this))`). This drains the liquid bToken inventory that backs the bToken's own pool, leaving the pool unable to transfer bTokens to users (e.g., swaps, staking withdrawals), effectively bricking the pool until the vault is exited.

RECOMMENDATION

When setting a vault for a reserve that is also a bToken with an active pool, only deposit the excess reserves above `totalBTokens + totalStaked + credit.accounts[address(this)].collateral` (or the pool's required liquid inventory), not the full balance.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/16a40bcf8d713d6a1c1ccc06dd8788c372a96933/src/components/BController.sol#L95>

MEDIUM **M-12**

ACKNOWLEDGED

Convexity parameter is being overrelaxed

CATEGORY Math

LOCATION CurveLib.sol#L306

REVIEW Main Review

DESCRIPTION

The protocol adapts the curve's convexity (n) using `CurveLib.computeMinimumConvexityExp` so a single curve can be consistent with both the current state and the historical max state `(maxCirc, maxReserves)`. The invariant is:

$$K = B * (x/c)^n$$

$$B = y - BLV * c$$

$$x = \text{totalSupply} - c$$

Taking logs:

$$\ln K = \ln B + n(\ln x - \ln c)$$

Solving for n using the current state and the max state (and eliminating K) gives:

$$n = (\ln B_{\text{max}} - \ln B_{\text{now}}) / [(\ln x_{\text{now}} - \ln c_{\text{now}}) - (\ln x_{\text{max}} - \ln c_{\text{max}})]$$

Here, `x_now` is the remaining supply (`supply = totalSupply - circ`). The code uses `ln(totalSupply)`, which inflates the denominator and yields a **smaller n** than required^{**}. This flattens the curve, so the resulting curve no longer passes through the max point as intended. Practically, it underprices buys in later states and weakens the intended safety margin above BLV.

```
int256 lnSupply = (_params.totalSupply).toInt256().lnWad();
```

RECOMMENDATION

Use `supply` instead

```
function computeMinimumConvexityExp(
CurveParams memory _params,
uint256 _maxReserves,
uint256 _maxCirc
) internal pure returns (uint256 convexityExp_) {
int256 lnMaxBuffer = (_maxReserves - _params.BLV.mulWad(_maxCirc)).toInt256().lnWad();
```

M-12 Convexity parameter is being overrelaxed

CONTINUED

```
int256 lnBuffer = (_params.reserves - _params.BLV.mulWad(_params.circ)).toInt256().lnWad();
int256 lnMaxCirc = _maxCirc.toInt256().lnWad();
- int256 lnSupply = _params.totalSupply.toInt256().lnWad();
+ int256 lnSupply = _params.supply.toInt256().lnWad();
int256 lnCirc = _params.circ.toInt256().lnWad();
int256 lnMinSupply = (_params.totalSupply - _maxCirc).toInt256().lnWad();
convexityExp_ = FixedPointMathLib.divWad(
(lnMaxBuffer - lnBuffer).toUint256(),
(lnMaxCirc + lnSupply - lnCirc - lnMinSupply).toUint256()
);
}
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1768594959614/commit/e8e0c327944f5d4b9b927eb79def6ae90968b898>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/abc71eba553151253c7eb914fe38c79cc0789d6d/src/libraries/CurveLib.sol#L299>

MEDIUM **M-13**

ACKNOWLEDGED

Underreported supply drains shared reserves

CATEGORY Gaming
LOCATION `src/components/BController.sol#L145`
REVIEW Main Review

DESCRIPTION

`setBTokenDeployment()` allows setting `State.pool(_bToken).totalSupply` to a value lower than the actual `BToken.totalSupply()`. A pool creator can then hold more `BToken` units than the pool accounting recognizes, stake them via `BStaking.deposit()`, and inflate `getMaxBorrow()` / `borrow()` because credit capacity is computed from collateral and `blvPrice` without capping to accounted supply or available reserves. The `borrow()` payout is sourced from shared reserve custody via `VaultLib.giveReserves()`, so the attacker can drain reserves funded by other pools.

The risk is further amplified if there is an upgradeability logic. For example, a token with a fixed `totalSupply` may later introduce a logic for minting and successfully drain the pool reserves.

RECOMMENDATION

Cap collateralization to accounted circulating supply, e.g., enforce `totalStaked + _amount <= State.pool(_bToken).totalSupply - State.pool(_bToken).totalBTokens` in `BStaking.deposit()`.

Also prefer allowing tokens with fixed total supply.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/72b8f7687d1401442d11814d6b96586e42b80d8c/test/unit/BCredit.t.sol#L1135-L1194>

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/77094087b38cb7ccd0a76dd572714a6fc31bade1/src/components/BStaking.sol#L81>

LOW **L-01**

ACKNOWLEDGED

Swaps bricks when convexityExp > 2e18

CATEGORY	DoS
LOCATION	MakerLib()
REVIEW	Main Review

— DESCRIPTION

`_initialActivePrice` can push the curve into the non quadratic `convexityExp > 2` regime at initialization

And that regime immediately hits a code bug path due to a units mismatch introduced during `initialize()`

If `_initialActivePrice` is high enough, `MakerLib.initialize()` sets `maker.convexityExp > 2e18`

For `convexityExp > 2e18`, every swap goes through `_updateCurveConvexity()`

`_updateCurveConvexity()` compares native decimal pool totals against WAD normalized max values that were stored during initialization

```
require(pool.totalReserves > maker.maxReserves);
```

With common reserves like USDC 6 decimals, this comparison will always fail, so the first buy and any trade that tries to push circulating > `maker.maxCirc` reverts, DoSing buys / growth for that pool

— RECOMMENDATION

Consider normalizing current `circulating` and `totalReserves` to WAD before comparing and assigning

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/506fea7bfff1c90f6bd860f068629f98006109c9e/src/libraries/MakerLib.sol#L258>

LOW **L-02**

ACKNOWLEDGED

Unaccounted dust desync reserves accounting

CATEGORY	Unexpected Behavior
LOCATION	MakerLib
REVIEW	Main Review

— DESCRIPTION

Both `exactIn` and `exactOut` paths perform an extra reserve transfer called "dust" after executing the swap. This transfer pulls additional reserves from the user (`buyExactIn` pulls the difference up to `amountIn`; `sellExactOut` pulls back any overage above target) and increases the vault's actual reserves. However, these extra reserves are not reflected in `pool.totalReserves` nor distributed via MakerLib's accounting. This creates a persistent mismatch between actual reserves and the curve's tracked reserves, underpricing the pool and leaking fees away from the configured fee distribution logic.

— RECOMMENDATION

We could incorporate the dust into MakerLib accounting

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/2fcc5507549a0b0b98b4e7ff3e4964cc1f6e557f/src/components/BSwap.sol#L117>

LOW **L-03**

ACKNOWLEDGED

Pool creation may fail due to underflow

CATEGORY DoS
LOCATION CurveLib.sol#L240-266
REVIEW Main Review

DESCRIPTION

`CurveLib.computeInitialCurveParams()` decides whether it can use the `n = 2` branch based on `quadraticPrice >= _initialActivePrice`. However, `quadraticPrice` is computed with `divWadUp` (rounding up). This makes `quadraticPrice` a ceiling, not the exact value. The inequality required for the `n = 2` BLV formula to be safe is:

```
quadraticPrice * supply * circ <= 2 * reserves * totalSupply
```

That inequality holds in exact math, but a 1-wei upward rounding in `quadraticPrice` can flip it when multiplied by very large `supply * circ`. As a result, the code can enter the `n=2` branch even though the exact inequality fails, and the subtraction

```
2 * reserves * totalSupply - initialActivePrice * circ * supply
```

underflows and reverts.

This results in unexpected DOS for some pools.

RECOMMENDATION

You can:

1. Make the `quadraticPrice` calculation equivalent to the one in `computeActivePrice()`

```
- uint256 quadraticPrice = _initialBLV + FixedPointMathLib.divWadUp(  
- buffer.mulWad(2e18).mulWad(_params.totalSupply),  
- _params.supply.mulWad(_params.circ)  
- );  
+ uint256 quadraticPrice = _initialBLV + FixedPointMathLib.fullMulDiv(  
+ buffer,  
+ uint256(2e18).mulWad(_params.totalSupply),  
+ _params.supply.mulWad(_params.circ)  
+ );
```

1. Round the `convexityExp` up so `n < 2` is avoided

L-03 Pool creation may fail due to underflow

CONTINUED

```
uint256 priceDelta = _initialActivePrice - _initialBLV;  
- uint256 term = FixedPointMathLib.fullMulDiv(priceDelta, _params.supply,  
_params.totalSupply);  
- convexityExp_ = FixedPointMathLib.fullMulDiv(term, _params.circ, buffer);  
+ uint256 term = FixedPointMathLib.fullMulDivUp(priceDelta, _params.supply,  
_params.totalSupply);  
+ convexityExp_ = FixedPointMathLib.fullMulDivUp(term, _params.circ, buffer);
```

As an additional safety measure, you can assert the computed `convexityBuffer` is not below 2.

```
if (convexityExp < 2e18) revert()
```

If this behavior is not problematic, you can also revert with a clear reason instead of letting it underflow.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-fuzz-1768594995707/blob/e81dfea314afacb3137cc1791a3712e31fee7568/test/fuzzing/FoundryRound3.sol#L26-L43>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/7e2716658107ac392d3d241ddcc03f349314c3bb/src/libraries/CurveLib.sol#L231>

LOW **L-04**

ACKNOWLEDGED

Unused code in quote functions

CATEGORY	Superfluous Code
LOCATION	BSwap.sol#L212-222 BSwap.sol#L312-318
REVIEW	Main Review

DESCRIPTION

`BSwap.quoteBuyExactIn` and `BSwap.quoteSellExactOut` both compute an initial `delta` estimate (based on `reservesIn / priceWithFee`), cap it, and then never use it. Each function immediately calls its respective solver (`_solveBuy` / `_solveSell`), which performs an independent bracketed binary search. As a result, the entire `delta` block in both functions is dead code: it does not affect outputs, gas usage in the solver, or correctness.

This unused computation makes the code harder to reason about and may mislead reviewers into thinking the solver is seeded with this value. It also adds unnecessary computation and leads to increased gas costs.

RECOMMENDATION

Remove the unused `delta` computation blocks in both `quoteBuyExactIn` and `quoteSellExactOut`.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/9d25aed78e413b5c675d1b408cba61a7811c17fa/src/components/BSwap.sol#L195>

LOW **L-05**

ACKNOWLEDGED

Smaller swaps reverting due to fee rounding to 0

CATEGORY DoS
LOCATION —
REVIEW Main Review

— DESCRIPTION

`MakerLib.swapTokens` computes swap fees in WAD via `CurveLib.computeSwap` and then denormalizes them to native reserve decimals with `NormalizeLib.denormalizeWad`. For low-decimal reserve tokens (e.g., 6 decimals), small swaps can produce a non-zero `feesWad` that still rounds down to 0 in native units. The current guard `require(feesReceived_ != 0 && deltaUserReserves_ != 0, InvalidOutput());` treats that rounding artifact as an error and reverts. This creates a denial-of-service on small trades: the curve returns a valid quote and the user pays rounded-against amounts, but the swap fails solely because the denormalized fee is zero. It also makes it harder to sell tokens as the pool supply increases.

If the revert were simply relaxed without any additional logic, the rounding loss would silently divert fees away from the intended recipients. The fee is already embedded in the swap curve output, so users would still pay it, but the fee would remain inside pool reserves instead of being recorded as creator/protocol/staker fees. That is an accounting drift and changes fee distribution semantics.

— RECOMMENDATION

Accumulate fee "dust" in WAD and only realize it once it reaches at least 1 native unit, while relaxing the revert to check that the WAD-level fee is non-zero. This prevents small-trade DoS caused by rounding.

Add a WAD remainder bucket to pool state:

```
--- a/src/libraries/StateLib.sol
+++ b/src/libraries/StateLib.sol
@@ struct Pool {
uint128 creatorFeePct; // Share of profit to creator
uint128 creatorClaimable; // Total fees claimable by creator
uint128 protocolClaimable; // Total fees claimable by protocol
uint128 pendingYield; // Fees pending distribution to stakers
```

L-05 Smaller swaps reverting due to fee rounding to 0

CONTINUED

```
+ // Accumulated fee dust in WAD (kept until it reaches 1 native unit)
+ uint256 feeRemainderWad;
}
```

Accumulate fee dust and relax the revert to check `feesWad` instead of `feesReceived_`:

```
--- a/src/libraries/MakerLib.sol
+++ b/src/libraries/MakerLib.sol
@@ function swapTokens(...)
- feesReceived_ = NormalizeLib.denormalizeWad(feesWad, rDec);
- require(feesReceived_ != 0 && deltaUserReserves_ != 0, InvalidOutput());
+ // Accumulate sub-unit fee dust in WAD, only realize once it reaches 1 native unit
+ uint256 feesWadTotal = feesWad + pool.feeRemainderWad;
+ feesReceived_ = NormalizeLib.denormalizeWad(feesWadTotal, rDec);
+ pool.feeRemainderWad -= NormalizeLib.normalizeWad(feesReceived_, rDec);
+
+ require(feesWad != 0 && deltaUserReserves_ != 0, InvalidOutput());
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1768594959614/commit/99857d084f0198d78dfb74dd8c70fdbe38877eff>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/a1037957095eb560f887477cc1bde5339667e61d/src/libraries/MakerLib.sol#L155>

LOW **L-06**

ACKNOWLEDGED

Remove final correction from buy/sell paths

CATEGORY Best Practices
LOCATION BSwap.sol#L408-411 BSwap.sol#L486-490
REVIEW Main Review

DESCRIPTION

At the end of the `_solveBuy()` and `_solveSell()` functions, there is a correction logic which modifies the delta in a way that makes the user lose more value.

```
_solveBuy():  
  
// CONSERVATIVE: Reduce to ensure reserve-side swap is always worse than token-side.  
// Scale reduction based on decimal precision difference to handle 8-decimal reserves.  
uint256 reduction = FixedPointMathLib.max(1, delta_ / 10_000_000); // 0.00001% reduction (10x  
tighter)  
if (delta_ > reduction) delta_ -= reduction;  
  
_solveSell():  
  
// CONSERVATIVE: Increase to ensure reserve-side swap is always worse than token-side.  
// Scale increase based on decimal precision difference to handle 8-decimal reserves.  
uint256 maxDelta = _pool.totalSupply - _pool.totalBTokens;  
uint256 increase = FixedPointMathLib.max(1, delta_ / 10_000_000); // 0.00001% increase (10x  
tighter)  
if (delta_ + increase <= maxDelta) delta_ += increase;
```

Since both of the functions are wrappers around `CurveLib.computeSwap()`, no further adjustment to the delta is needed. The binary search approach is already not in favor of the user due to the dust they have to pay, adjusting the delta makes them incur unnecessary losses.

RECOMMENDATION

Remove the final correction logic from the two functions.

RESOLUTION

Status: Acknowledged.

L-06 Remove final correction from buy/sell paths

CONTINUED

<https://github.com/0xBaseline/mercury/blob/d04e852b3db75c963883c9004b410704d5713765/src/components/BSwap.sol#L322>

LOW **L-07**

ACKNOWLEDGED

Asymmetrical validation for buy path

CATEGORY	Validation
LOCATION	BSwap.sol
REVIEW	Main Review

— DESCRIPTION

`BSwap.quoteBuyExactIn()` limits certain buys, where the reserves received are more than 95% of the current reserves.

```
if (reservesInWad > p.reserves * 95 / 100)
    revert AmountExceedsLiquidity();
```

However, this check is not present in `buyTokensExactOut()`, allowing users to bypass it by using the other function.

— RECOMMENDATION

Consider whether the check is needed, since it limits the token in, not the token out. If it's - add it to the other function as well, otherwise remove it.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Incorrect event emission for hook swaps

CATEGORY Events
LOCATION MakerLib.sol#L206
REVIEW Main Review

DESCRIPTION

`MakerLib._recordSwap()` emits the `Swap` event with `msg.sender` as the user.

```
emit Swap(  
  _bToken,  
  msg.sender,  
  CurveLib.computeActivePrice(getCurveParams(_bToken)),  
  maker.blvPrice,  
  _deltaCirc,  
  _deltaUserReserves,  
  _feesReceived,  
  liquidityFee  
);
```

For swaps performed through the `BHook` contract, `msg.sender` will always be the `Relay` itself. This leads to incorrect value emitted and makes it harder to track swaps offchain.

RECOMMENDATION

If emitting the correct user is important, you can introduce permissioned wrapper functions around `BSwap` that forward the sender.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/8aee8c26cc685862d19bc8ecebfc038be889bde9/src/libraries/MakerLib.sol#L207>

LOW **L-09**

ACKNOWLEDGED

solveBuy() can revert due to overflow

CATEGORY Math
LOCATION BSwap.sol#L354
REVIEW Main Review

DESCRIPTION

In `BSwap._solveBuy()`, the code multiplies `totalBTokens * 99`. If `totalBTokens * 99 > type(uint128).max`, the operation will revert (checked arithmetic in Solidity ≥ 0.8), causing swaps/liquidity operations that reach this branch to fail even though the values may be otherwise valid for the surrounding logic. This creates a hard cap on `totalBTokens` and can brick functionality once the pool grows beyond `type(uint128).max / 99`.

RECOMMENDATION

Cast `totalBTokens` to `uint256` before multiplying.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/d5c2211c2b73543b9ca1291a53d2c38cd0d2caab/src/components/BSwap.sol#L335>

LOW **L-10**

ACKNOWLEDGED

Buy fee path underflow can revert swaps

CATEGORY DoS
LOCATION CurveLib.sol#L171
REVIEW Main Review

DESCRIPTION

In `CurveLib._computeFee` (buy branch), `newBufferDown` is recomputed and then used in `fee_ = absDelta.mulWadUp(marginalPremium) - (newBufferDown - buffer)`.

The whole recomputation path is rounded down (including division and multiplication steps, as well as `powWad`), which can produce `newBufferDown < buffer` even while processing a buy.

When that happens, `(newBufferDown - buffer)` underflows and the swap reverts, causing DoS for otherwise valid buy paths.

RECOMMENDATION

Prevent fee-path underflow by using a bounded buffer delta, e.g. `bufferIncrease = newBufferDown.zeroFloorSub(buffer)` before subtraction.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-fuzz-1768594995707/commit/25f2d413a176fc79798eff35673d4822f35520d8>

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/f716d5c08fe72cade906cd017f938a3ce2c3650f/src/libraries/CurveLib.sol#L153>

LOW **L-11**

ACKNOWLEDGED

Buffer can grow during sells

CATEGORY Math
LOCATION CurveLib.sol#L184
REVIEW Main Review

DESCRIPTION

The sell path can violate expected buffer monotonicity due to compounded upward rounding. In `CurveLib.computeInvariant()`, `lastInvariant(K)` is rounded up.. In `CurveLib.computeSwap()`, sell quotes derive `newBuffer` from `lastInvariant` using upward rounding as well. This is usually done to round in favor of the protocol and require more reserves from the user. However, the new buffer may become greater than the current one, especially for smaller sells. This will result in negative `invariantDelta` and the sell will actually behave like a buy for the reserves. In `CurveLib._computeFee()` (sell branch), the fee path performs an unsigned subtraction equivalent to $(\text{buffer} - \text{_newBuffer})$ before IL-premium adjustment. When `\_newBuffer` exceeds `buffer`, this subtraction underflows and the swap reverts, leading to unexpected failures, and inconsistent sell behavior.

RECOMMENDATION

If this is acceptable, consider reverting with appropriate error if the new buffer increases during swaps. Otherwise, the buffer calculation should be reconsidered.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-fuzz-1768594995707/blob/bad29d1b81e99f01d1c14c8593b9e0e6fad9ede0/test/fuzzing/FoundryRound3.sol#L17>

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/f716d5c08fe72cade906cd017f938a3ce2c3650f/src/libraries/CurveLib.sol#L167>

LOW **L-12**

ACKNOWLEDGED

Hook sweep can revert batched swaps

CATEGORY DoS
LOCATION src/libraries/MakerLib.sol:226
REVIEW Main Review

DESCRIPTION

`MakerLib._updateAccounting()` calls `SweepLib.sweep()` at the start of every swap. When swaps are routed through the hook, the input token is recorded as an outstanding balance and settled later by the router. If an integrator batches multiple hook swaps in the same unlock and defers settlement until the end (a common v4 routing pattern), the second swap will try to sweep the first swap's unpaid outstanding balance and revert when `poolManager.take` cannot transfer funds that haven't been settled yet. This makes multi-swap batching through the hook incompatible unless each swap is settled before the next one, reducing composability and potentially breaking routers that rely on open deltas.

RECOMMENDATION

Either:

- Skip `SweepLib.sweep()` if the pool manager is unlocked and the hook still has an unpaid delta, and defer sweeping to a later action.
- Require integrators to settle after each hook swap when batching (and document the constraint).

PROOF OF CONCEPT https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/poc_hook_sweep_multiswap_revert/test/unit/HookSweepMultiSwapRevert.poc.t.sol

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/d288a2edbb0a24a14e663617539d99b89c9f035c/src/libraries/SweepLib.sol#L63>

LOW **L-13**

ACKNOWLEDGED

Recorded reserves drift due to vault operations

CATEGORY Math
 LOCATION —
 REVIEW Main Review

DESCRIPTION

When a swap occurs, `CurveLib` computes the amount of `Δreserves` and returns it as `deltaResWad`. If the direction is buying and a vault is set for that reserve, the received tokens will be immediately deposited to that vault which will lead to a precision loss. To mitigate that, `MakerLib` overcharges the user in such a way that after the deposit, the system will have its balance increased with at least `Δreserves`

```
deltaUserReserves_ = -(
VaultLib.convertAssetsToDeposit(
State.pool(_bToken).reserve,
NormalizeLib.denormalizeWadUp(uint256(-deltaResWad), rDec)
).toInt256());
```

Then `deltaUserReserves_` is passed to `recordSwap()`, where `totalReserves` will be increased with that value and the user will have to pay the exact same amount. Because of this, the rounding error that's covered by the user will be attributed to `totalReserves` even though this amount will be lost.

For example, let's say the invariant computes `Δy` assets needed. In order to get these assets we have to deposit `Δy + ε` to the vault. The code will record an increase in `totalReserves` of `Δy + ε` even though the system received only `Δy`.

RECOMMENDATION

You can add a new parameter to `_recordSwap()` called `rawAmount` and use it to pass the value before the vault deposit if the direction is a buy.

```
_recordSwap(_bToken, _deltaCirc, deltaUserReserves_, feesReceived_, deltaResWad < 0 ? -
int256(NormalizeLib.denormalizeWadUp(uint256(-deltaResWad), rDec)) : int256(0));
```

`0` is being used as a default value when the swap is sell, so `_recordSwap` will handle that case.

L-13 Recorded reserves drift due to vault operations

CONTINUED

```
if (rawAmount == 0) {  
    rawAmount = _deltaUserReserves;  
}
```

Finally, `_updateAccounting()` will use the raw amount instead.

```
uint256 liquidityFee = _updateAccounting(_bToken, _deltaCirc, rawAmount, _feesReceived);
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1768594959614/commit/562233e94004f0b9df9198c5e15571dbdbacb882>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/d8185686cab6b4d1acc3c199078f21c89ef95ead/src/libraries/MakerLib.sol#L145>

LOW **L-14**

ACKNOWLEDGED

Hook sweep can under-credit vault

CATEGORY Math

LOCATION src/libraries/VaultLib.sol:301

REVIEW Main Review

DESCRIPTION

When a reserve has a vault set, the funds from swaps are deposited to it during settlement. Swaps use `convertAssetsToDeposit()` to charge the user an amount that would result in `targetAssets` after the deposit. With the same vault ratio at quote and deposit ($r_1 = r_2$), the rounding buffer (`VAULT_ROUNDING_BUFFER = 2`) ensures `previewRedeem(previewDeposit(X)) >= targetAssets`, so the pool does not end up under-credited.

For swaps performed through `BHook` the user amount is computed at time t_1 using `convertAssetsToDeposit()`, but the actual vault deposit happens later when `sweep` executes. This introduces a ratio gap ($r_1 = tS_1/tA_1$ at quote, $r_2 = tS_2/tA_2$ at deposit), so the original rounding guarantee no longer necessarily holds.

Let `a` be the target assets. The rounding pipeline is:

$$\mathrm{previewWithdraw}(a) = \lceil a \cdot r_1 \rceil = a \cdot r_1 + \epsilon, \quad \epsilon \in [0, 1]$$

$$\mathrm{assetsNeeded} = \mathrm{previewRedeem}(\mathrm{previewWithdraw}(a) + 2) = \lfloor a + \frac{\epsilon + 2}{r_1} \rfloor = a + \lfloor \frac{\epsilon + 2}{r_1} \rfloor$$

$$\mathrm{sharesMinted} = \lfloor \mathrm{assetsNeeded} \cdot r_2 \rfloor = \mathrm{assetsNeeded} \cdot r_2 - \delta, \quad \delta \in [0, 1]$$

$$\mathrm{creditedAssets} = \lfloor \frac{\mathrm{sharesMinted}}{r_2} \rfloor = a + \lfloor \frac{\epsilon + 2}{r_1} \rfloor - \lceil \frac{\delta}{r_2} \rceil$$

$$\mathrm{creditedAssets} < a \iff \lfloor \frac{\epsilon + 2}{r_1} \rfloor - \lceil \frac{\delta}{r_2} \rceil < 0$$

Case $r_2 \geq 1$ (share price ≤ 1 asset): $\lceil \delta / r_2 \rceil$ is 0 or 1. Loss is only possible when $\lfloor (\epsilon + 2) / r_1 \rfloor = 0$ and $\delta > 0$, which happens when $(\epsilon + 2) / r_1 < 1$ (i.e., $r_1 > 2 + \epsilon$). Using a buffer of one full asset's worth of shares (`previewWithdraw(1) ≈ ceil(r1)`) guarantees $\lfloor (\epsilon + \text{buffer}) / r_1 \rfloor \geq 1$, so the loss condition cannot hold when $r_2 \geq 1$.

L-14 Hook sweep can under-credit vault

CONTINUED

Case $r_2 < 1$ (share price > 1 asset): $\text{ceil}(\delta/r_2)$ can exceed 1, so even with a larger buffer a loss is still technically possible if r_2 drops enough between t_1 and t_2 .

— RECOMMENDATION

For Case $r_2 \geq 1$, you can increase the share buffer in `convertAssetsToDeposit` to one full asset's worth of shares, and keep it at least `VAULT_ROUNDING_BUFFER`: use `max(previewWithdraw(1), VAULT_ROUNDING_BUFFER)` instead of a fixed +2. This removes the loss condition and improves accuracy under rate drift.

Case $r_1 < 1$ cannot be fully solved because r_2 is not known at t_1 , but it can be improved by frequent sweeps.

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/4dd60579772b6eda212e23b8dd2d8ff55ff71fa0/src/libraries/VaultLib.sol#L305>

LOW **L-15**

ACKNOWLEDGED

Harvest buffer not retained on vault exit

CATEGORY Math
LOCATION src/libraries/VaultLib.sol:217
REVIEW Main Review

DESCRIPTION

`HARVEST_BUFFER` is used in `getHarvestableYield()` and in `switchVaults()`'s pre-check to ensure there is enough yield to cover precision errors on vault entry/exit. However, `exitVault()` computes `yield_ = redeemed_ - depositedReserves` and immediately pays out the full yield to the fee recipient. This means the buffer is not retained during a vault exit. As a result, any rounding losses are no longer offset by the intended buffer and are effectively borne by the pool's reserves instead of being absorbed by retained yield.

RECOMMENDATION

When exiting a vault, retain the buffer by subtracting `HARVEST_BUFFER` from the payout: compute `yield_ = (redeemed_ - depositedReserves).zeroFloorSub(HARVEST_BUFFER)` before distributing fees. Then update the `switchVaults` check to require `yield_ != 0` so the original safety threshold remains intact.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/80487f76ba426b9a304602be1f90d64a943a4edd/src/libraries/VaultLib.sol#L214>

LOW **L-16**

ACKNOWLEDGED

Harvest buffer can be dynamically adjusted

CATEGORY Suggestion
LOCATION VaultLib.sol#L20
REVIEW Main Review

DESCRIPTION

`HARVEST_BUFFER` in `VaultLib.sol` is meant to absorb potential rounding losses from vault interactions. Currently it's hardcoded to 100 wei, but may not be enough if vault shares are expensive.

A mixed approach of using both a hardcoded buffer and dynamically adjusted one can be made. You can measure losses when a vault is entered, withdrawals are performed or idle funds are deposited and accumulate the losses to a dynamic buffer. Then, when using buffer logic you can utilize `max(HARVEST_BUFFER, dynamicBuffer)`.

Currently, `depositIdleReserves()` tries to use a part of the available yield in order to cover the loss from the deposit. However, the only thing it achieves is to ensure there is more extractable yield than the realized loss. Once this yield is extracted, the proxy contract will still bear a loss. With the dynamic buffer approach, you can add the `precisionError` to it, which will ensure that precision loss is actually not later withdrawn as yield and enough funds remain in the contract in order to sufficiently back `totalReserves`.

RECOMMENDATION

Consider implementing the dynamic buffer approach, especially in `depositIdleFunds()`.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/3d0ac7f711a6ca7b5a1d3da0136fb8a8ebc962b2/src/libraries/VaultLib.sol#L153>

LOW **L-17**

ACKNOWLEDGED

Pool reserves overstated on createPool()

CATEGORY	Rounding
LOCATION	—
REVIEW	Main Review

— DESCRIPTION

`createPool()` sets `pool.totalReserves` to `params.initialPoolReserves + params.initialDebt` before collecting reserves. It then calls `takeReserves()` to pull `params.initialPoolReserves` from the caller. If a vault is already allocated for the reserve (likely when the same reserve is shared across pools), `takeReserves()` deposits into the vault via `depositToVault()`, which can round down and credit fewer assets than the amount transferred. This creates an initial shortfall where `pool.totalReserves` is overstated relative to the vault-redeemable value.

— RECOMMENDATION

When collecting initial reserves in `createPool()`, charge the user with `convertAssetsToDeposit()` so the vault credits the intended `params.initialPoolReserves`. This keeps `pool.totalReserves` consistent with the actual vault-credited assets.

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/88cf1c0019a1aaa0309f921fa726a5fba86b62d3/src/components/BFactory.sol#L201>

LOW **L-18**

ACKNOWLEDGED

Duplicate claim leaves enable griefing

CATEGORY Trust Assumptions
LOCATION src/components/BCredit.sol:211
REVIEW Main Review

DESCRIPTION

`claimCredit()` is permissionless and `_processClaim()` only tracks `credit.claimed[user]` per user. If a user appears twice in the Merkle root with different amounts, any third party can call `claimCredit()` with the smaller leaf first, setting `credit.claimed[user] = true` and permanently blocking the larger claim via `BCredit_AlreadyClaimed()`. This turns a root-construction mistake into a griefing vector where the user is forced to accept the lower allocation.

RECOMMENDATION

Either document that users can claim only once and this must be enforced when generating the Merkle root, or change the tracking to be per-leaf instead of per-user so multiple leaves for the same user can be claimed without griefing.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/6b7c7ccd878732e8308469d04ad0cff87de10b42/src/components/BCredit.sol#L172>

LOW **L-19**

ACKNOWLEDGED

Leverage fee charged on max borrow

CATEGORY Math
LOCATION BCredit.sol:286
REVIEW Main Review

DESCRIPTION

In `leverage()`, `getBorrowForCollateral()` computes `borrowAmount` and `fee` from `_totalCollateral` using `State.meta().originationFee`. After `buyTokensExact0ut()` returns `reservesIn`, the code reduces `debt_` by the unused `borrowAmount`, but `fee` is not adjusted and is still distributed via `State.pool(_bToken).distributeFees()`. When `reservesIn` is lower than `borrowAmount`, the effective fee rate becomes `fee / reservesIn`, which is higher than the intended `originationFee` and can overcharge `leverage()` users while inflating `pendingYield` relative to the realized swap cost.

RECOMMENDATION

Recompute `fee` from the realized borrow (`reservesIn`) or from the final `debt_` after the refund, and use that recomputed `fee` in `distributeFees()`.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/0f1a32826dc29d0e84f6e772e36e7a64b8319772/src/components/BCredit.sol#L315>

LOW **L-20**

ACKNOWLEDGED

Ratio rounding favors users on buys and sells

CATEGORY Rounding
LOCATION CurveLib.sol#L95
REVIEW Main Review

DESCRIPTION

In `CurveLib.computeSwap()`, both buy and sell branches compute `ratio = x1 / c1` using `divWadUp()`. Rounding the ratio up makes `powWad()` larger and reduces `newBuffer`. For buys, this lowers the required reserves, so users pay less and `effectivePrice` can fall below `activePrice()` as observed in `FoundryRound3::test_replay()`. For sells, the same rounding reduces `newReserves`, increasing user payout. This is opposite to the documented protocol-favorable rounding intent.

RECOMMENDATION

Use `divWad()` for the `ratio` in both branches to round against the user, and update the comments to match the rounding policy.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-fuzz-1768594995707/blob/be4478e36b786f9d3ae5b06faf65b14273833da9/test/fuzzing/FoundryRound3.sol#L17>

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/7361f1f8ea9c1e2b4651f568088d34fb61184c70/src/libraries/CurveLib.sol#L91>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Missing guard causes sell side reverts

CATEGORY Informational
LOCATION `computeSwap()`
REVIEW Main Review

DESCRIPTION

The BUY branch protects `powWad` and `expWad` from overflowing by enforcing `n` times `ln(ratio)` less than or equal to approximately `135e18`. The SELL branch computes the same `powWad` term for `ratio` equal to `x1` divided by `c1` and greater than 1, but omits the guard. For large `x1` divided by `c1` or high `convexityExp`, `int256(ratio).powWad(n)` overflows due to the `expWad` limit and reverts. This makes otherwise valid sell quotes or executions revert, causing a denial of service in high convexity or deep sell scenarios.

RECOMMENDATION

Mirror the BUY branch overflow guard in the SELL branch before calling `powWad`

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/f55c8ffaf890db97a526edd82e175f63410d4c6f/src/libraries/CurveLib.sol#L93>

INFORMATIONAL **I-02**

ACKNOWLEDGED

Excess repayment funds not refunded

CATEGORY Informational
LOCATION BCredit.sol
REVIEW Main Review

— DESCRIPTION

Both `repay()` and `repayWithNative()` compute the actual debt to repay via `previewRepay/repay` (which caps to the user's remaining debt), but then unconditionally pull the full `reservesIn/msg.value` from the payer. Any excess over what is needed to repay debt is not refunded and is not credited to the user

— RECOMMENDATION

We could only pull the exact amount required to cover debt, if the caller supplied a higher amount refund the difference by using `convertAssetsToDeposit`

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/954261a56243fae55f66605fe311ab839aba4202/src/components/BCredit.sol#L147>

INFORMATIONAL I-03

ACKNOWLEDGED

Repay preview revert when debt is 0

CATEGORY Suggestion
LOCATION BCredit.sol
REVIEW Main Review

DESCRIPTION

In `previewRepay()`

```
debtToRepay_ = min(_debtAmount, _account.debt);  
collateralRedeemed_ = uint256(_account.collateral).mulDiv(debtToRepay_, _account.debt);
```

If `_account.debt == 0`, we divide by zero even though `debtToRepay_` will be 0

So `previewRepay`, `previewRepayExactDebt`, and `repay` will revert

RECOMMENDATION

Consider handling zero debt early

if (`_account.debt == 0`) return (0, 0);

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/df3b92f283e5e33359f1370ea58c7ab9149ab10c/src/components/BCredit.sol#L544>

INFORMATIONAL **I-04**

ACKNOWLEDGED

harvestYield has a return but doesn't return it

CATEGORY Informational
LOCATION BController.sol
REVIEW Main Review

DESCRIPTION

harvestYield declares a return but doesn't return it

```
function harvestYield(ERC20 _reserve) external permissioned nonReentrant returns (uint256) {  
    uint256 yield_ = _reserve.harvestYield();  
    emit Harvested(address(_reserve), yield_);  
}
```

RECOMMENDATION

Consider returning yield_

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/440850b0504dfaa1535635df1b26766ae3f5c788/src/components/BController.sol#L87>

INFORMATIONAL I-05

ACKNOWLEDGED

Missing upper bound blocks pool creation

CATEGORY Validation
LOCATION BFactory.sol
REVIEW Main Review

DESCRIPTION

In `createBToken`, `totalSupply` is only lower bounded, but the protocol later downcasts circulating supply to `uint128`

The user choose `_totalSupply` and the factory stores it as the pool supply

`_validateBTokenParams()` only enforces a minimum

```
require(_totalSupply >= MIN_TOTAL_SUPPLY, TotalSupplyTooLow());
```

The rest of the system assumes that circulating supply can fit into `uint128`

But during `createPool()`, it does

```
maker.maxCirc = NormalizeLib  
.normalizeWad(pool.totalSupply - pool.totalBTokens, bDec)  
.toUint128();
```

If `_totalSupply` is large enough such that

`pool.totalSupply - pool.totalBTokens > type(uint128).max`

then `toUint128()` reverts, and the pool cannot ever be initialized for that `bToken`.

So a user can successfully deploy a token via `createBToken()` but can not create a pool for it

RECOMMENDATION

Cap `_totalSupply` so it can not exceed `uint128` by adding an upper bound in `_validateBTokenParams`

RESOLUTION

Status: Acknowledged.

I-05 Missing upper bound blocks pool creation

CONTINUED

<https://github.com/0xBaseline/mercury/blob/547384e00c259fa03c5a37fb290bbcf5fbdd7983/src/components/BFactory.sol#L303>

INFORMATIONAL **I-06**

ACKNOWLEDGED

isExecutor() can use dirty address

CATEGORY	Best Practices
LOCATION	Component.sol#L64
REVIEW	Main Review

DESCRIPTION

`Component._isExecutor(address)` accepts an address and returns if this address is an executor. To do so, it hashes the address together with the mapping storage slot. However, it doesn't clean the upper 12 bytes of the address. This is fine for the current implementation, but if code changes and the address provided has dirty bytes, the returned value may be incorrect.

```
function _isExecutor(address _user) internal view returns (bool isExecutor_) {
    assembly {
        mstore(0x00, _user)
        // executorExpiry mapping is at slot 5
        mstore(0x20, 5)
        // find the slot for the user in the mapping
        let slot := keccak256(0x00, 0x40)
        // load the expiration timestamp and compare to current timestamp
        isExecutor_ := gt(sload(slot), timestamp())
    }
}
```

RECOMMENDATION

Consider cleaning the upper 12 bytes of `_user`

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/46292a7c215c8bda575c43e2783fd4bf9703bcbf/src/Component.sol#L65>

INFORMATIONAL **I-07**

ACKNOWLEDGED

Relay routes are unnecessary

CATEGORY Superfluous Code
LOCATION Relay.sol#L66-84
REVIEW Main Review

DESCRIPTION

A `_setRelayRoutes()` function was introduced to the `Relay` contract and is invoked in its constructor. It sets `address(this)` as a route for each function selector of the `Relay` contract.

```
function _setRelayRoutes() internal {  
  // state getters  
  routes[this.admin.selector] = address(this);  
  routes[this.pendingAdmin.selector] = address(this);  
  routes[this.getComponentForLabel.selector] = address(this);  
  routes[this.routes.selector] = address(this);  
  routes[this.executorExpiry.selector] = address(this);  
  // view functions  
  routes[this.getLabels.selector] = address(this);  
  routes[this.isExecutor.selector] = address(this);  
  // non view functions  
  routes[this.transferAdmin.selector] = address(this);  
  routes[this.acceptAdmin.selector] = address(this);  
  routes[this.addExecutor.selector] = address(this);  
  routes[this.removeExecutor.selector] = address(this);  
  routes[this.executeActions.selector] = address(this);  
}
```

Doing this is unnecessary, because if the first 4 bytes of the calldata match any of the functions, they will be executed as a normal EVM call, and the `fallback` won't even be entered.

RECOMMENDATION

Remove the `_setRelayRoutes()` function.

I-07 Relay routes are unnecessary

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL I-08

ACKNOWLEDGED

Incorrect ceil comments

CATEGORY Best Practices
LOCATION BCredit.sol#L558-559
REVIEW Main Review

DESCRIPTION

The comments in `BCredit.getBorrowForCollateral()` say the divisions there are using `ceil`, while it's actually `floor` that's happening.

```
uint256 debtWad = blv.mulWad(collateralWad); // ceil for obligation
uint256 debt = NormalizeLib.denormalizeWad(debtWad, rDec); // ceil
```

RECOMMENDATION

Remove the comments.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/c102fe997ee42f10134c27b3eddddfef33cff9ebf/src/components/BCredit.sol#L559>

INFORMATIONAL I-09

ACKNOWLEDGED

Redundant mulWad () in active price computation

CATEGORY Superfluous Code
LOCATION CurveLib.sol#L62
REVIEW Main Review

DESCRIPTION

`CurveLib.computeActivePrice()` computes the premium to be added to `BLV`:

```
uint256 premium = FixedPointMathLib.fullMulDiv(  
_params.reserves - _params.BLV.mulWad(_params.circ),  
_params.convexityExp.mulWad(_params.totalSupply),  
_params.supply.mulWad(_params.circ).mulWad(WAD)  
);
```

On the last line of the code snippet, we can see `mulWad(WAD)`. Because `mulWad` is equivalent to $x * y / WAD$, the end result will be $x * WAD / WAD = x$, which makes the operation unnecessary.

RECOMMENDATION

Delete the last `mulWad()`

```
uint256 premium = FixedPointMathLib.fullMulDiv(  
_params.reserves - _params.BLV.mulWad(_params.circ),  
_params.convexityExp.mulWad(_params.totalSupply),  
- _params.supply.mulWad(_params.circ).mulWad(WAD)  
+ _params.supply.mulWad(_params.circ)  
);
```

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/90c44036d1e570144a4566d4ab8c7d7f14ea76a5/src/libraries/CurveLib.sol#L109>

INFORMATIONAL **I-10**

ACKNOWLEDGED

Unreachable edge cases in computeSwap()

CATEGORY	Suggestion
LOCATION	CurveLib.sol#L73-86
REVIEW	Main Review

DESCRIPTION

`CurveLib.computeSwap()` has code that handles specific edge cases - full buys and sells.

```
// Edge case: buying from zero circulation
if (_params.circ == 0) {
    return _computeZeroCircSwap(_params, uint256(_deltaCirc));
}
uint256 c1 = (_params.circ.toInt256() + _deltaCirc).toUint256();
// Edge case: selling to zero circulation
// User only receives BLV value (floor price), buffer becomes protocol fee
if (c1 == 0) {
    uint256 blvValue = _params.BLV.mulWad(_params.circ);
    uint256 receipt = blvValue.mulWad(WAD - _params.swapFee);
    return (int256(receipt), _params.reserves - receipt);
}
```

However, the invariant curve $K = B \times (x/c)^n$ is not defined for $c = 0$. Because of that `CurveLib.computeInvariant()` won't allow the pool to ever reach a state where $c = 0$, therefore these codepaths should never be executed.

RECOMMENDATION

Reconsider whether the edge cases code is needed.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/e21011f5a7214ea3182a1849523d9651b1f008db/src/libraries/MakerLib.sol#L191>

INFORMATIONAL **I-11**

ACKNOWLEDGED

Binary search loop can break earlier

CATEGORY Gas Optimization
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

The binary search loops in `_solveBuy()` and `_solveSell()` compare midpoints against desired targets, but they don't `break` in case `midPoint == target`.

This may cause unnecessary iterations and increased gas cost.

RECOMMENDATION

Consider breaking if the target is found.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/9106bf33c9ba74fb98e20147f47b0235ece0bf4b/src/components/BSwap.sol#L361>

INFORMATIONAL I-12

ACKNOWLEDGED

Dead Pool.shareBalance state field

CATEGORY	Superfluous Code
LOCATION	—
REVIEW	Main Review

DESCRIPTION

`State.Pool` declares `uint128 shareBalance` at , but this field is never read from or written to anywhere in the codebase. A search for `pool.shareBalance` and assignments to `shareBalance` in `src/` returns no usages.

At the same time, vault share accounting is implemented through `State.meta().allocations[_reserve]` and `BLens.shareBalance()` reads `alloc.vault.balanceOf(address(this))` rather than pool-level storage. This indicates `Pool.shareBalance` is dead state.

RECOMMENDATION

Consider removing the field

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-13**

ACKNOWLEDGED

K decreases due to computation inconsistencies

CATEGORY Rounding
LOCATION MakerLib.sol#L195
REVIEW Main Review

DESCRIPTION

When `convexityExp` and `BLV` are unchanged, `MakerLib._recordSwap` recomputes the invariant and enforces strict non-decrease.

`computeInvariant` depends on `powWad` (approximation via `ln/exp`) plus fixed-point rounding (`divWad`, `divWadUp`, `mulWadUp`, `divWadUp`).

This can introduce small negative numerical drift between the stored invariant and recomputed invariant, causing swap reverts even when the observed delta is tiny relative to invariant magnitude.

RECOMMENDATION

Keep the check, but document this as an intentional precision-related limitation.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-fuzz-1768594995707/commit/d9db73fd8eef82da7f2d4c4fbe4a1fed8d42bd70>

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Vaults are a security risk

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Users can freely deploy their BTokens and provide `reserves` for backing to the Baseline Relay and start using its features to launch their token.

Currently the executor of the `Relay` contract can choose a vault where the reserves will be deposited in order to generate additional yield. This feature is not riskfree. A compromised executor can provide any vault of their choice, including a fake address which just pulls the funds out of the contract.

Even if the executor acts honestly, if the vault is not properly monitored, the relay contract can lose its reserves, either due to security risks in the vault itself or economic losses.

— RECOMMENDATION

Be aware of the security risks that come with having the vault design and consider if changes are necessary.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-15**

ACKNOWLEDGED

Unused previewTakeReserves() helper

CATEGORY	Superfluous Code
LOCATION	src/libraries/VaultLib.sol:263
REVIEW	Main Review

DESCRIPTION

The helper `previewTakeReserves` is defined in `VaultLib` but has no call sites in the codebase. It duplicates `convertDepositToAssets` behavior and adds unused surface area, which can confuse future refactors and audits.

RECOMMENDATION

Remove `previewTakeReserves` or add a concrete call site that relies on it. If it's intended for future use, add a short comment explaining where/why it should be used.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-16**

ACKNOWLEDGED

Outdated refund ordering comment

CATEGORY Documentation
LOCATION src/components/BCredit.sol
REVIEW Main Review

DESCRIPTION

The comment in `deleverage()` states that issuing the refund before `unlockCollateral()` prevents it from being treated as yield during `_sync()`. Current yield accounting in `BStaking` is driven by `pendingYield`, `claimableYield`, and `staking.totalStaked`, and is not affected by the order of `giveReserves()` versus `unlockCollateral()`. As a result, the comment does not reflect the present behavior.

```
// refund should be given before unlocking collateral so it's not treated as yield during
_sync
if (refund_ > 0) State.pool(_bToken).reserve.giveReserves(msg.sender, refund_, true);
```

RECOMMENDATION

Remove the comment.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/d8d6c25ef1f605b0b274c4c02277c28c1cdfc1b7/src/components/BCredit.sol#L393>

INFORMATIONAL **I-17**

ACKNOWLEDGED

Invariant error lacks drop context

CATEGORY Best Practices
LOCATION —
REVIEW Main Review

DESCRIPTION

`MakerLib._recordSwap()` reverts with `InvariantDecreased()` when `CurveLib.computeInvariant()` returns a smaller value, but the error carries no values. This makes it difficult to distinguish between precision drift and meaningful invariant drops during fuzzing, monitoring, and debugging.

RECOMMENDATION

Add parameters to `InvariantDecreased()` (e.g., `prevInvariant` and `newInvariant`) and revert with `revert InvariantDecreased(prevInvariant, newInvariant)` so tooling can assess the magnitude and context of the drop.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/1eafb8857d83238187112cd137efd98833057b84/src/libraries/MakerLib.sol#L69>

INFORMATIONAL **I-18**

ACKNOWLEDGED

Check computed blv against book price

CATEGORY Best Practices
LOCATION src/libraries/CurveLib.sol:268
REVIEW Main Review

DESCRIPTION

In `computeInitialCurveParams()`, the final validation checks `_initialBLV` against `bookPrice` via `require()`, even though the function may compute a different `BLV_` in the quadratic branch. `BLV_` is still bounded by `bookPrice` when `_initialActivePrice > bookPrice`, so this is not a functional bug, but the check is semantically tied to the computed output and would be clearer and more future-proof if applied to `BLV_` directly.

RECOMMENDATION

Replace the final check with `require(BLV_ <= bookPrice, InvalidBLVPrice())`

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/d41af439767db76e0263400a11ffdf503edc87dc/src/libraries/CurveLib.sol#L261>

INFORMATIONAL **I-19**

ACKNOWLEDGED

Lack of BToken parameters validation

CATEGORY	Validation
LOCATION	BController.sol#L145-151
REVIEW	Main Review

DESCRIPTION

`BController.setBTokenDeployment()` doesn't run the logic in `_validateBTokenParams()`, which means a token with unexpected configuration may be added to the system.

RECOMMENDATION

Consider validating the token parameters.

RESOLUTION

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

K drops due to overcharged fee

CATEGORY	Rounding
LOCATION	MakerLib.sol#L152
REVIEW	Remediation Review 1

— DESCRIPTION

In response to L-05 the feesReceived variable is rounded up.

```
feesReceived_ = NormalizeLib.denormalizeWadUp(feesWad, rDec);
```

This resolves the problem with the swaps reverting when swap fee cannot be represented as a whole unit of the reserve token. However, that same `feesReceived_` variable is passed to `_recordSwap()` and will be accounted as a fee at the expense of the reserves. Let's look at an example where the reserve token is in 8 decimals.

- invariantDelta = -40000000000000000221497
- feesWad = 40000000000000000218505
- feesReceived_ = 400000000001
- deltaResWad = `invariantDelta - feesWad` = -440000000000000000440002

This means the user will pay 4400000000001 reserves of which 4000000000001 are fees. The rest 40000000000000 go to the backing reserves. If we normalize this number to 18 decimals, we can see that $40000000000000000000 < 40000000000000000000221497$ (invariantDelta).

Once the swap is done, the actual `totalReserves` in the contract are less than expected, therefore the calculated invariant will also be lower than it should. This case becomes more problematic in the region of the curve where more of 95% of the total supply is sitting in the pool because there isn't liquidity fee to compensate for the invariant drop. In result, the curve will use the outdated `lastInvariant` which is bigger than the actual invariant and further will be blocked.

— RECOMMENDATION

Consider implementing the original recommendation of L-05

PROOF OF CONCEPT https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/d0s_sells_decreasing_invariant/test/unit/DOS_Sells_POC.t.sol

H-01 K drops due to overcharged fee

CONTINUED

— RESOLUTION

Status: Acknowledged.

HIGH **H-02**

ACKNOWLEDGED

Sell side buffer rounding breaks swap

CATEGORY DoS
LOCATION CurveLib.sol
REVIEW Remediation Review 1

DESCRIPTION

Sell side IL fee math can revert for a huge range of sell sizes

computeSwap() calls _computeFee()

_computeFee() sell branch $_deltaCirc < 0$ contains two requires

```
require(buffer > _newBuffer, InvalidFeeForSwap());  
require(marginalBufferDelta < bufferDecrease, InvalidFeeForSwap());
```

IL fee $bufferDecrease - marginalBufferDelta$ is combined with swap fee on premium portion on sells

For certain reachable curve states, especially when

BLV is extremely close to book price reserves / circ

the remaining buffer is tiny $buffer = reserves - BLV * circ \approx 0$

and convexity is moderate or high $convexityExp > 2e18$, but it can even happen for $2e18$

The sell fee math becomes fragile due to inconsistent rounding between

newBuffer being computed rounded up in computeSwap

```
uint256 newBuffer = fullMulDivUp(lastInvariant, 1e18, pow(ratio,n));
```

marginalPremium and marginalBufferDelta being computed rounded down or mixed in the SELL branch

```
uint256 marginalPremium = fullMulDiv(newBuffer, );  
uint256 marginalBufferDelta = absDelta.mulWad(marginalPremium);
```

and then enforcing strict inequalities

$buffer > newBuffer$

$marginalBufferDelta < bufferDecrease$

H-02 Sell side buffer rounding breaks swap

CONTINUED

when buffer is tiny, the up rounding of newBuffer very easily makes

newBuffer = buffer so bufferDecrease = 0 then revert

or bufferDecrease so small that marginalBufferDelta >= bufferDecrease then revert

So the pool can end up with a minimum sell size, and that minimum can become large hundreds of thousands of tokens

and alongside sell swap DoS, it affects deleverage also

so borrowers cannot reduce debt by selling collateral

Therefore a pool can become effectively non tradable in one direction for most sizes

Below is a curve state where:

selling 100 tokens reverts with InvalidFeeForSwap

selling 175301 tokens succeeds

meaning all sells smaller than about 175k tokens are impossible

```
// All parameters in WAD units
totalSupply = 1_000_000e18
pool.totalBTokens supply = 18_698e18 circ = totalSupply - supply = 981_302e18
bookPrice = reserves/circ = 4e18
BLV = 4e18 - 19 only 19 wei below book price
buffer = reserves - BLV * circ = 18_644_738 tiny
convexityExp = 10e18
swapFee = 0.01e18
```

In that state

_deltaCirc = -100e18 revert InvalidFeeForSwap

_deltaCirc = -175_301e18 OK

The revert happens because the sell branch tries to compute

IL fee = bufferDecrease - marginalBufferDelta

then adds swap fee on premium but the IL fee step is guarded by

```
require(marginalBufferDelta < bufferDecrease, InvalidFeeForSwap());
```

and rounding makes that condition fail for a huge range of _deltaCirc

H-02 Sell side buffer rounding breaks swap

CONTINUED

The buy branch explicitly compensates for rounding by recomputing a different buffer `newBufferDown` for fee calculation

```
uint256 newBufferDown = fullMulDiv(lastInvariant, pow(), 1e18);
```

But the sell branch does not do an analogous rounding safe recomputation, and instead uses `newBuffer` that was computed using `fullMulDivUp` rounded up, then enforces the strict require

<https://gist.github.com/GuardianAudits/30d68bde1f917caf36e7f45036e55476>

— RECOMMENDATION

make the sell side fee math use a rounded down buffer, instead of using `computeSwap()` `newBuffer` which is `fullMulDivUp` ceil, this will help, but might be not enough, consider redesigning this area

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/c9b80084a8b56929bd02b102d537fc4c67efa0f1/src/libraries/CurveLib.sol#L166>

HIGH **H-03**

ACKNOWLEDGED

Inverse pow rounding inflates buy side fees

CATEGORY Rounding
 LOCATION CurveLib.sol
 REVIEW Remediation Review 1

DESCRIPTION

buy side inverse pow rounding can turns most of the trade into creator claimable fees

In computeSwap(), the main swap math uses

```
uint256 ratio = x1.divWad(c1);
uint256 newBuffer = fullMulDivUp(lastInvariant, WAD, powWad(ratio, n));
```

Then the buy branch of _computeFee() recomputes a rounded down buffer using the reciprocal ratio

```
uint256 newBufferDown = FixedPointMathLib.fullMulDiv(
  _p.lastInvariant,
  int256(c1.divWad(x1)).powWad(_p.convexityExp.toInt256()).toUint256(),
  WAD
);
```

That is dangerous because for buys where the pool still holds more than half the supply after the trade $x1 > c1$, we have

$x1 / c1 > 1$ in computeSwap(), main path is fine

but $c1 / x1 < 1$ inside _computeFee(), powWad(c1/x1, n) can round to 0

Example

If post trade $x1 / c1 = 3$

and convexityExp = 50e18

Then

```
main path uses 3^50, which is still finite
fee path uses (1/3)^50 ≈ 1.4e-24
In WAD math, (1/3)^50 rounds to 0
```

That makes

H-03 Inverse pow rounding inflates buy side fees

CONTINUED

```
newBufferDown = 0;
bufferIncrease = 0;
```

So buy IL fee becomes

```
fee ≈ marginalBufferDelta + BLV swap fee
```

Instead of

```
fee = marginalBufferDelta - trueBufferIncrease + BLV swap fee
```

So the code reclassifies almost the entire premium spend as fee

This lets high convexity pools massively overcharge buys without reflecting that in the visible swapFeePct also if we looked how that fee is handled afterward

```
liquidityFee_ = _feesReceived - 50% of _feesReceived;
if (pool.totalBTokens >= 95% of totalSupply) {
  liquidityFee_ = 0;
}
pool.distributeFees(_bToken, _feesReceived - liquidityFee_);
```

When the pool still holds at least 95% of supply, common early pool state, 100% of the inflated fee is distributed, not retained

So a pool creator can

1. create a high convexityExp pool,
2. keep inventory high (totalBTokens >= 95% totalSupply),
3. let users buy,
4. have _computeFee() silently explode the fee,
5. claimPoolFees() and siphon those reserves

<https://gist.github.com/GuardianAudits/a39a8e6b1afac439eec25ade8c4039ed>

— RECOMMENDATION

we need to not recompute newBufferDown via the reciprocal c1/x1, reuse _newBuffer and round it down

Inside _computeFee() buy branch, replace

```
uint256 newBufferDown = FixedPointMathLib.fullMulDiv(
  _p.lastInvariant,
  int256(c1.divWad(x1)).powWad(_p.convexityExp.toInt256()).toUint256(),
```

H-03 Inverse pow rounding inflates buy side fees

CONTINUED

```
WAD  
);
```

with this, same math as computeSwap(), just rounded down

```
uint256 newBufferDown = FixedPointMathLib.fullMulDiv(  
_p.lastInvariant,  
WAD,  
int256(x1.divWad(c1)).powWad(_p.convexityExp.toInt256()).toUint256()  
);
```

This removes the powWad(c1/x1, n) underflow to zero path on buys when $x1 > c1$ and makes newBufferDown the floor version of the exact buffer computation used for _newBuffer,

so the premium cannot be silently reclassified into creator claimable fees

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/415a3e39fd95907cf2c7a144f3b9e4394089e398/src/libraries/CurveLib.sol#L149>

HIGH **H-04**

ACKNOWLEDGED

Convexity reset removes buy side price impact

CATEGORY Math
 LOCATION CurveLib.sol
 REVIEW Remediation Review 1

DESCRIPTION

User triggered convexity relaxation erases buy side price impact, so split buys are massively underpriced

On a buy that pushes the pool to a new all time high book price, `_translateConvexity()` Path 1 computes a new exponent from the pre trade price

```
uint256 prevPrice = CurveLib.computeActivePrice(_prev);
uint256 floor = FixedPointMathLib.fullMulDiv(
  prevPrice - _params.BLV,
  _params.supply.mulWad(_params.circ),
  actualBuffer.mulWad(_params.totalSupply)
);
```

But `computeActivePrice()` is

$$P = BLV + \text{buffer} * n * \text{totalSupply} / (\text{supply} * \text{circ})$$

So after the trade, if we set

$$n_{\text{new}} = (\text{prevPrice} - BLV) * \text{supply} * \text{circ} / (\text{buffer} * \text{totalSupply})$$

then the new post trade marginal price becomes

$$P_{\text{after}} = BLV + \text{buffer} * n_{\text{new}} * \text{totalSupply} / (\text{supply} * \text{circ}) = \text{prevPrice}$$

So the code is literally snapping the post trade price back to the pre trade price

`_recordSwap()` prices the current buy using the old, steeper curve first, then lowers `convexityExp` after accounting. That means

1 - Trade 1 is charged under old `n`

2 - State is then rewritten so the next trade starts from the old price again

H-04 Convexity reset removes buy side price impact

CONTINUED

3 - Repeating small buys lets the attacker walk through a convex curve at an almost flat marginal price

4 - maxCirc/maxReserves are updated first, so the attacker can staircase this repeatedly until n collapses to $2e18$

That's order splitting underpricing issue, example state

```
BLV = 1
convexityExp = 6e18
totalSupply = 1000
pool inventory / circulating = 500 / 500
reserves = 1500
swap fee = 0.3%
```

A single buy of 70 tokens costs about 9367 reserve units.

But if the attacker splits it into 7 buys of 10

total cost is only about 2021

convexityExp ratchets from 6 down to about 2.04

the post trade active price stays ~25 for the first several chunks instead of ratcheting up

That is not normal convex curve behavior. It means fragmentation itself manufactures a huge discount

So attacker can exploit this by

1 - Find a pool with convexityExp > $2e18$ and premium gate satisfied

2 - Split a target buy into many small buys

3 - Each buy triggers Path 1 and cheapens the next state

<https://gist.github.com/GuardianAudits/afbf268d14324d977507450fb4f7360a>

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/cd76e271a4d4083beb5dfd2c4b609dc2>

— RESOLUTION

Status: Acknowledged.

HIGH **H-05**

ACKNOWLEDGED

Round trip swaps enable reserves drain

CATEGORY Math
LOCATION MakerLib.sol
REVIEW Remediation Review 1

DESCRIPTION

convexityExp can be permanently ratcheted downward with tiny sell / buy back round trips, even if the attacker returns the pool to essentially the same inventory split

recordSwap() calls translateCurve(_bToken, prev) after every swap

when convexityExp > 2e18, _translateCurve() routes to _translateConvexity()

translateConvexity() has two paths (currentBookPrice >= maxBookPrice and < maxBookPrice) and both can only decrease convexityExp

nothing ever increases convexityExp, so it becomes a path dependent, global one way state

Allowing this attack

1. Repeatedly do a tiny round trip

sellTokensExactIn()

buyTokensExactOut() (buy back)

2. The sell can push price below maxBookPrice (relaxing convexityExp via the below path), and the buy-back can cross back above (relaxing it again via the above path using prevPrice)

3. After each loop, totalBTokens / circ are ~unchanged (fees aside), but convexityExp is permanently lower

4. Repeat until convexityExp collapses toward 2e18 at trivial cost

5. Then execute a single large sellTokens() or deleverage(), with a flatter curve, the protocol pays out materially more reserves than it would on an untouched curve

This differs from splitting a large sell, because here the attacker can pre soften the curve cheaply with micro round trips, end with nearly the same position, and monetize later in one big swap

In the PoC attacker spends ~2 reserves to unlock ~38 extra reserves on the later dump for example, which can be repeated to drain the buffer

H-05 Round trip swaps enable reserves drain

CONTINUED

<https://gist.github.com/GuardianAudits/ed0e25a10793a0f761f7709c72464002>

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/cd76e271a4d4083beb5dfd2c4b609dc2>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

computeSwap can DoS swaps

CATEGORY DoS
LOCATION CurveLib.sol
REVIEW Remediation Review 1

DESCRIPTION

In computeSwap()

```
uint256 ratio = x1.divWadUp(c1);  
require(  
ratio <= WAD || _params.convexityExp.mulWad(uint256(int256(ratio).lnWad())) <= 135e18,  
TradeExceedsLimit()  
);  
uint256 newBuffer = FixedPointMathLib.fullMulDivUp(  
_params.lastInvariant,  
WAD,  
int256(ratio).powWad(_params.convexityExp.toInt256()).toUint256()  
);
```

when $\text{ratio} < 1e18$ ($x1 < c1$), the require does not compute / guard anything,

and execution proceeds to the powWad() denominator

powWad() in solady is implemented as

```
return expWad(( lnWad(x) * y) / WAD);
```

and expWad() explicitly returns 0 for sufficiently negative inputs

```
if (x <= -41446531673892822313) return r; // r = 0
```

So when $\text{ratio} < 1$, $\text{lnWad}(\text{ratio})$ is negative, and convexityExp is large

the exponent $(\text{lnWad}(\text{ratio}) * \text{convexityExp}) / \text{WAD}$ can become $\leq -41.4465e18$

then expWad() returns 0, therefore powWad(ratio, convexityExp) returns 0

and fullMulDivUp(denominator = 0) reverts

that's a DoS reachable from pool state supply/circ and a high convexityExp

let $n = \text{convexityExp} / 1e18$

M-01 computeSwap can DoS swaps

CONTINUED

we get the failure when

```
ln(ratio) * n <= -41.4465  
=  
ratio <= exp(-41.4465 / n)
```

Examples

```
If n = 50, threshold is ratio <= exp(-0.8289) = 0.436
```

Meaning once post trade $x1/c1$ drops below ~0.436,

powWad rounds to zero and the swap math reverts

```
If n = 20, threshold is ratio <= exp(-2.0723) = 0.126
```

```
If n = 10, threshold is ratio <= exp(-4.1446) = 0.0159
```

So in high convexity pools, this can happen at non extreme circulation ratios

once the pool reaches a state where for a buy the resulting $x1/c1$ is below that threshold,

then all paths that call computeSwap for buys, buyTokensExactIn / buyTokensExactOut / leverage, will revert, until the state is moved back into a safe area via enough sells increasing x/c

— RECOMMENDATION

This issue happens because we divide by powWad(ratio,n) when $\text{ratio} < 1e18$

we can do the same math, but remove the need of powWad()

Instead of `newBuffer = K / ( ratio^n )`

we can compute the same thing as

```
newBuffer = K * ( invRatio^n )
```

Mathematically these are identical

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

computeNextBLV can DoS swaps

CATEGORY DoS
LOCATION CurveLib.sol
REVIEW Remediation Review 1

DESCRIPTION

computeNextBLV() computes

```
prevCirc = _params.totalSupply - _prevSupply
```

then uses prevCirc squared in the denominator

```
uint256 prevCirc = _params.totalSupply - _prevSupply;  
uint256 penalty = fullMulDivUp(  
_params.supply.mulWad(_params.supply).mulWad(prevCirc).mulWad(prevCirc) // prevCirc^2  
);
```

prevCirc can be 0, whenever the previous pool state had

prevSupply = totalSupply (pool holds all bTokens), so prevCirc = totalSupply - prevSupply = 0

that can be triggerable DoS for swaps that cross the 95% threshold,

because _recordSwap() calls computeNextBLV() whenever

pool is in the update BLV path, maker.convexityExp = 2e18

and pool ownership is below the safety threshold

```
if (pool.totalBTokens < pool.totalSupply.mulWad(BUFFER_SAFETY_THRESHOLD)) {  
// if convexityExp == 2e18:  
maker.blvPrice = CurveLib.computeNextBLV(getCurveParams(_bToken), prev.supply,  
prev.reserves);  
}
```

So If someone sells enough bTokens so the pool ends up with pool.totalBTokens = pool.totalSupply, that makes prevCirc = 0

Next, someone tries to do a single buy large enough that after the buy

pool.totalBTokens < 0.95 * totalSupply (circulating becomes > 5%)

M-02 computeNextBLV can DoS swaps

CONTINUED

That swap triggers the BLV update path and calls computeNextBLV() with prevCirc = 0

And the swap revert

a small buy < 5% can move off circ = 0 and avoid the revert on the next swap

But any swap that tries to move from circ = 0 to pool has < 95% of supply in one trade will revert

This affects also leverage() / deleverage() flows because they go through _recordSwap() too

— RECOMMENDATION

At the start of computeNextBLV()

```
uint256 prevCirc = _params.totalSupply - _prevSupply;  
// If previous circulation was zero, there is no meaningful penalty term  
// So we can keep BLV unchanged  
if (prevCirc == 0) return _params.BLV;
```

This makes the circ = 0 to big buys transitions safe

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/efa622844950de3fa8a74c67240e68d3afe6e956/src/libraries/CurveLib.sol#L278>

MEDIUM **M-03**

ACKNOWLEDGED

Yield distribution can be violated

CATEGORY Rewards
LOCATION BStaking.sol
REVIEW Remediation Review 1

DESCRIPTION

The contract is trying to drip rewards out over time

It uses lastUpdated as the clock to measure how much time passed, and tokensPerSecond as the speed of dripping rewards

_sync() only persists tokensPerSecond and lastUpdated when accumulator changes,

but getAccumulator() can and does change tokensPerSecond even when no yield is distributed newYield = 0

That creates a state where

staking.tokensPerSecond gets stuck at an old non zero value, doesn't decay toward 0 when pendingYield = 0

staking.lastUpdated also gets stuck, timeElapsed grows large

That happens because sync() discards tokensPerSecond decay unless accumulator changed

```
(uint256 accumulator, uint256 newYield, uint256 tokensPerSecond) = getAccumulator(_bToken);  
if (accumulator != staking.accumulator) {  
    pool.pendingYield -= newYield.toUint128();  
    staking.claimableYield += newYield.toUint128();  
    staking.tokensPerSecond = tokensPerSecond.toUint128();  
    staking.accumulator = accumulator;  
    staking.lastUpdated = block.timestamp.toUint32();  
}
```

So if accumulatorIncrease = 0, happens when pendingYield = 0, then

staking.tokensPerSecond is not updated

staking.lastUpdated is not updated, so timeElapsed keeps growing

the next time pool.pendingYield becomes non zero again,

M-03 Yield distribution can be violated

CONTINUED

`yield = min(tokensPerSecond * timeElapsed, pendingYield)` collapses to `pendingYield`, meaning all pending yield becomes distributable in one `_sync()`, and therefore claimable immediately

this is MEV friendly. whoever can be the first to trigger `_sync()` after `pendingYield` refills gets to decide who is staked at the exact instant the entire bucket is pushed into `claimableYield/accumulator`

This breaks the intended reward drip model

Preconditions for this

1- `pendingYield` is 0 for a period

2- nobody calls a staking function that runs `sync()` for some time, so `staking.lastUpdated` doesn't advance and `tokensPerSecond` decay isn't persisted,

3- later `pendingYield` becomes > 0 , from swaps, borrow, and someone triggers `sync()`

<https://gist.github.com/GuardianAudits/445dde235979efc5e0b7e9fc35769a39>

— RECOMMENDATION

If this is acceptable consider acknowledging it

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/1b7518ed8a253384faead6d56c37e84928704a76/src/components/BStaking.sol#L213>

MEDIUM **M-04**

ACKNOWLEDGED

Reopen buy bypasses stored invariant

CATEGORY Logical Error
LOCATION CurveLib.sol
REVIEW Remediation Review 1

DESCRIPTION

In `computeZeroCircSwap()`

When a pool is in `circ = 0` and someone reopens it with a buy, the code does

```
uint256 ratio = _deltaCirc.divWadUp(x1);  
uint256 newBuffer = FixedPointMathLib.fullMulDivUp(  
  _p.lastInvariant,  
  int256(ratio).powWad(_p.convexityExp.toInt256()).toUint256(),  
  WAD  
);
```

If the reopen buy is small enough, $\text{ratio} < 1$, and for sufficiently large `convexityExp` the WAD fixed point result of

```
powWad(ratio, convexityExp)
```

rounds down to 0

In the normal swap path, that same kind of rounding results in the division by zero DoS issue

But here, in `computeZeroCircSwap`, the zero is used as a multiplier, so the trade does not revert. Instead

```
newBuffer = 0  
bufferReserves = 0
```

and the reopen buy is charged only

```
payment = _deltaCirc * BLV * (1 + swapFee)
```

So the buyer gets reopened supply at pure BLV + fee, while the invariant implied premium is silently discarded

a user can push a pool into `circ = 0` by selling all circulating bTokens through the special `c1 = 0` branch in `computeSwap`

M-04 Reopen buy bypasses stored invariant

CONTINUED

after that, `recordSwap()` does not refresh `lastInvariant`, because it only recomputes the invariant when

```
pool.totalBTokens < pool.totalSupply * 95%
```

at `circ = 0`, the pool holds 100% of supply, so `maker.lastInvariant` stays as a stale positive `K` from the pre zero state

that stale `K` is exactly what `computeZeroCircSwap()` is supposed to use to price the first reopen buy. But the WAD rounding zeroes it out

The rounding condition is

```
(deltaCirc / x1) ^ convexityExp < 1e-18
```

with `convexityExp = 50e18` for example, a reopen buy below about 30.4% rounds to 0 and triggers the issue

So in high convexity pools, an attacker can reopen a large chunk of supply at floor pricing

The first reopen buyer can acquire supply far cheaper than the stored invariant requires

That matters because those `bTokens` can then be sold later once the pool is reopened, or used to capture upside while the pool's premium was never paid on entry

So the premium encoded in `lastInvariant` is effectively bypassed

<https://gist.github.com/GuardianAudits/4d947e3373396c6e391a01d385fe0ae5>

— RECOMMENDATION

we need to prevent the `powWad(ratio, convexityExp)` rounding to 0 from silently zeroing `newBuffer / premium` on reopen buys

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-05**

ACKNOWLEDGED

Non additive IL fee exploitable

CATEGORY Logical Error
LOCATION CurveLib.sol
REVIEW Remediation Review 1

DESCRIPTION

In `_computeFee()` / `computeSwap()`

For a sell, the code does

```
invariantDelta = BLV * Δ + bufferDecrease  
fee = (bufferDecrease - marginalBufferDelta) + premiumSwapFee  
so userOut = invariantDelta - fee
```

that simplifies to

```
userOut ≈ Δ * ( BLV + (1 - swapFee) * marginalPremium_after )
```

So the seller is effectively paid the post trade marginal premium across the whole trade, not the integral over the interval

for a buy, the same thing happens in reverse

the buyer pays roughly the post trade marginal premium across the whole trade, plus the BLV fee,

Instead of paying the integral over the interval

That means the fee model is path dependent,

one big sell gets paid using the worst marginal price on the path,

but many small sells get paid using a sequence of better marginals,

so splitting strictly improves execution.

Likewise, one big buy pays the worst marginal price on the path,

but many small buys pay a staircase of cheaper marginals,

so splitting strictly lowers cost.

therefore, the IL fee is not enforceable on chain, because a user can always partition the trade

M-05 Non additive IL fee exploitable

CONTINUED

a contract can loop `sellTokensExactIn()` or `buyTokensExactOut()` in one transaction and collapse the non linear fee toward zero

<https://gist.github.com/GuardianAudits/2debf9a39176280c73d7772b20e4f6c8>

— RECOMMENDATION

Consider replacing `_computeFee()` with additive fee

<https://gist.github.com/GuardianAudits/cd76e271a4d4083beb5dfd2c4b609dc2>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

TPS decay computed but never applied

CATEGORY Logical Error
LOCATION BStacking.sol
REVIEW Remediation Review 1

DESCRIPTION

In `getAccumulator()` the code compute

```
uint256 decayedTps = tokensPerSecond_.mulWad(_decayFactorWad(timeElapsed, timeToDistribute));
```

but then we update from the old stored TPS

```
if (targetTps > decayedTps) {  
    tokensPerSecond_ += gain;  
} else {  
    tokensPerSecond_ -= gain;  
}
```

It should really update from the decayed TPS, not the stale one

otherwise decay is only used for comparison, but not actually used as the new base rate

After long gaps `decayedTps` can go ~ 0 so `gain` goes ~ 0 , leaving TPS stale

because `getAccumulator()` computes `decayedTps` but uses `tokensPerSecond_` from the pre decay value and uses it for yield, so decay isn't applied to the stored TPS

RECOMMENDATION

apply decay to the TPS state not just the comparison

```
// getAccumulator()  
uint256 decayedTps = tokensPerSecond_.mulWad(_decayFactorWad(timeElapsed, timeToDistribute));  
tokensPerSecond_ = decayedTps; // use decayed TPS as the new base
```

RESOLUTION

Status: Acknowledged.

M-06 TPS decay computed but never applied

CONTINUED

<https://github.com/0xBaseline/mercury/blob/c9b80084a8b56929bd02b102d537fc4c67efa0f1/src/components/BStaking.sol#L258>

LOW **L-01**

ACKNOWLEDGED

Pool initialization may fail

CATEGORY Validation
LOCATION CurveLib.sol#L243
REVIEW Remediation Review 1

DESCRIPTION

The following requirement was added to the first branch in `CurveLib.computeInitialCurveParams()`

```
require(convexityExp_ > 2e18 && convexityExp_ <= MAX_CONVEXITY, InvalidConvexityExp());
```

Because of rounding during arithmetic operations, the resulting `convexityExp` may end up being exactly `2e18`. Since `2e18` is excluded from the allowed values, the pool initialization will fail.

RECOMMENDATION

Consider relaxing the check to allow the convexity to be `2e18`.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/56d7000fbf08ffa2d28c108abc0ae9ce0d968481/src/libraries/CurveLib.sol#L243>

LOW **L-02**

ACKNOWLEDGED

Impossible to default if resulting `circ == 0`

CATEGORY DoS
LOCATION BCredit.sol#L275
REVIEW Remediation Review 1

DESCRIPTION

On each call to `defaultSelf()` the `totalBTokens` are increased and therefore the circulating supply decreases. After that, `CurveLib.computeInvariant()` is executed. This call will revert if the resulting circulating tokens are 0, blocking the default from happening.

RECOMMENDATION

Consider not recomputing the invariant if the resulting `circ == 0`. In fact, you can modify `computeInvariant()` to return the old `K` if `circ == 0`.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/0b826f2fb5c37277c7779e1325cd175ec52fba00/src/libraries/CurveLib.sol#L46>

LOW **L-03**

ACKNOWLEDGED

Hook swap bypasses reserve dust accounting

CATEGORY Logical Error
LOCATION BHook.sol
REVIEW Remediation Review 1

DESCRIPTION

buyTokensExactIn() explicitly captures the dust difference between user supplied `_amountIn` and the curve's computed `actualCost`

```
uint256 actualCost = (-userDeltaReserves).toUint256();  
uint256 dustFee = _amountIn > actualCost ? _amountIn - actualCost : 0;  
if (msg.sender != address(this)) {  
    pool.reserve.takeReserves(msg.sender, dustFee);  
    FeeLib.distributeFees(pool, _bToken, dustFee);  
}
```

In the Uniswap-v4 hook `BHook.beforeSwap`, the hook calls `BSwap()` as a self-call, so inside `BSwap` we have `msg.sender == address(this)`. That means the `dustFee` capture block is skipped

So the protocol can end up with extra reserve tokens actually received (the exact-in amount) that are not reflected in per-pool accounting (`pool.totalReserves`, fee distribution, et.) because the dust capture path never ran

RECOMMENDATION

Replace the dust block in `buyTokensExactIn` with

```
uint256 dustFee = _amountIn > actualCost ? _amountIn - actualCost : 0;  
if (dustFee > 0) {  
    State.Pool storage pool = State.pool(_bToken);  
    if (msg.sender != address(this)) {  
        pool.reserve.handleIncoming(msg.sender, dustFee);  
    }  
    FeeLib.distributeFees(pool, _bToken, dustFee);  
}
```

L-03 Hook swap bypasses reserve dust accounting

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/179c7d8b07deec24a9e44a4c6e1ea51a411c9ab/src/components/BSwap.sol#L95>

LOW **L-04**

ACKNOWLEDGED

Hidden reserves in hook swaps

CATEGORY Logical Error
LOCATION BHook.sol
REVIEW Remediation Review 1

DESCRIPTION

In hook exact-output sells, BHook self-calls sellTokensExactOut,

so msg.sender = address(this) skips the dustFee clawback + distributeFees block.

But swapTokens / _updateAccounting() still updates pool.totalReserves using the full userDeltaReserves (the curve's actual received amount),

while BHook.beforeSwap only transfers the requested _amountOut,

leaving actualReceived - _amountOut sitting in the Relay untracked

RECOMMENDATION

Replace the dust block in sellTokensExactOut with

```
uint256 dustFee = actualReceived > _amountOut ? actualReceived - _amountOut : 0;
if (dustFee > 0) {
    State.Pool storage pool = State.pool(_bToken);
    if (msg.sender != address(this)) {
        pool.reserve.handleIncoming(msg.sender, dustFee);
    }
    FeeLib.distributeFees(pool, _bToken, dustFee);
}
```

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/179c7d8b07deec24a9e44a4c6e1ea51a411c9ab/src/components/BSwap.sol#L95>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Initial credits may be unclaimable

CATEGORY	Warning
LOCATION	BCredit.sol
REVIEW	Remediation Review 1

DESCRIPTION

New `getBorrowForCollateral()` checks were added when a pool with credit position is created and when user claims. However, `getBorrowForCollateral()` performs math operations which include rounding and because of that $f(x) == f(x1 + x2)$, where $x = x1 + x2$ is not guaranteed.

This can lead to valid states causing unclaimable credits. For example:

- `BLV = 0.05 ether`
- `initialCollateral = 1000 ether`
- `initialDebt = 50 ether`

The pool will be created successfully because `initialDebt == maxDebt`. However because of rounding issues some of the users may not be able to claim their part.

RECOMMENDATION

Run a local simulation of the maximum allowed debt per each user before creating a pool with credit positions. It's important to not use `getBorrowForCollateral()` for this simulation because the pool is not created and its values will be 0.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-02**

ACKNOWLEDGED

Pool creation cannot be paid with native tokens

CATEGORY Best Practices
LOCATION BFactory.sol
REVIEW Remediation Review 1

DESCRIPTION

`BFactory.createPool()` uses `handleIncoming` to charge the user the needed amount of reserve tokens. Usually, this function supports payment with native token to be converted to its wrapped version, but because `createPool()` is not payable, such payments are impossible.

RECOMMENDATION

If this feature is desired, consider adding `payable` to `createPool()`. Otherwise, acknowledge the issue

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/179c7d8b07deec24a9e44a4c6e1ea51a411c9ab/src/components/BFactory.sol#L145>

INFORMATIONAL **I-03**

ACKNOWLEDGED

Outdated comment

CATEGORY	Best Practices
LOCATION	BFactory.sol#L181-182
REVIEW	Remediation Review 1

DESCRIPTION

The comment at `BFactory.sol#L181-182` is outdated as there is no vault anymore.

```
// use the vault-credited amount (after rounding) for totalReserves  
pool.totalReserves = totalReserves.toUint128();
```

RECOMMENDATION

Remove the comment.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/179c7d8b07deec24a9e44a4c6e1ea51a411c9ab/src/components/BFactory.sol#L173>

INFORMATIONAL **I-04**

ACKNOWLEDGED

Lack of validation for creatorFeePct

CATEGORY	Validation
LOCATION	BController.sol#L110
REVIEW	Remediation Review 1

DESCRIPTION

`BController.modifyCreatorFeePct()` allows the executor to change the the creator fee for a given pool, but does not validate the new fee is in range (`<= 1e18`).

RECOMMENDATION

Consider adding the same validation as in `BFactory.createPool()`

```
require(params.creatorFeePct <= 1e18, InvalidCreatorFee());
```

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/179c7d8b07deeec24a9e44a4c6e1ea51a411c9ab/src/components/BController.sol#L111>

INFORMATIONAL I-05

ACKNOWLEDGED

Redundant function parameter

CATEGORY Superfluous Code
LOCATION BCredit.sol#L544
REVIEW Remediation Review 1

DESCRIPTION

The internal `BCredit._previewRepay()` function defines a `bToken` parameter that's never used.

RECOMMENDATION

Remove the `bToken` parameter.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/179c7d8b07deec24a9e44a4c6e1ea51a411c9ab/src/components/BCredit.sol#L544>

INFORMATIONAL **I-06**

ACKNOWLEDGED

previewRepay () should return debtToRepay

CATEGORY	Suggestion
LOCATION	BCredit.sol#L536
REVIEW	Remediation Review 1

— DESCRIPTION

The debt repayment flow was reworked to repay the exact amount sent by the user and revert if it's more than the active debt.

The public `previewRepay()` function now returns only `collateralRedeemed_`, but it still caps the debt that will be repaid to the maximum available amount.

```
_reservesIn = FixedPointMathLib.min(_reservesIn, account.debt);
```

This may mislead offchain integrators that use this function.

— RECOMMENDATION

Consider returning `_reservesIn` as a second parameter so it's known how much of the debt was repaid.

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/179c7d8b07deeec24a9e44a4c6e1ea51a411c9ab/src/components/BCredit.sol#L537>

INFORMATIONAL I-07

ACKNOWLEDGED

Redundant mulWad() in active price computation

CATEGORY Superfluous Code
LOCATION CurveLib.sol#L62
REVIEW Remediation Review 1

DESCRIPTION

`CurveLib.computeActivePrice()` computes the premium to be added to `BLV`:

```
uint256 premium = FixedPointMathLib.fullMulDiv(
    _params.reserves - _params.BLV.mulWad(_params.circ),
    _params.convexityExp.mulWad(_params.totalSupply),
    _params.supply.mulWad(_params.circ).mulWad(WAD)
);
```

On the last line of the code snippet, we can see `mulWad(WAD)`. Because `mulWad` is equivalent to $x * y / WAD$, the end result will be $x * WAD / WAD = x$, which makes the operation unnecessary.

RECOMMENDATION

Delete the last `mulWad`

```
uint256 premium = FixedPointMathLib.fullMulDiv(
    _params.reserves - _params.BLV.mulWad(_params.circ),
    _params.convexityExp.mulWad(_params.totalSupply),
    - _params.supply.mulWad(_params.circ).mulWad(WAD)
    + _params.supply.mulWad(_params.circ)
);
```

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

ACKNOWLEDGED

Unused errors

CATEGORY	Superfluous Code
LOCATION	BFactory.sol#L38-40
REVIEW	Remediation Review 1

DESCRIPTION

The following errors in `BFactory` are not used:

- `InsufficientPoolBTokens()`
- `InvalidBTokenDecimals()`
- `InvalidReserveDecimals()`

RECOMMENDATION

Remove the errors

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/179c7d8b07deec24a9e44a4c6e1ea51a411c9ab/src/components/BFactory.sol#L38>

INFORMATIONAL I-09

ACKNOWLEDGED

Comment mismatch on premium threshold

CATEGORY Documentation
LOCATION MakerLib.sol#L273
REVIEW Remediation Review 1

DESCRIPTION

In `MakerLib._translateConvexity()` the comment says "premium > 100% (activePrice > 2 × BLV)", but the code returns only when `premium < WAD`, so relaxation happens when `premium >= WAD`, i.e., `activePrice >= 2 × BLV` (subject to rounding). This is a comment-logic mismatch that can mislead reviewers about the exact gating condition.

RECOMMENDATION

Align the comment with the code behavior.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-10**

ACKNOWLEDGED

Confusing plural used for variable naming

CATEGORY Best Practices
LOCATION BCredit.sol#L259
REVIEW Remediation Review 1

DESCRIPTION

The credit account of the user in `defaultSelf()` is named `accounts` instead of `account` which may be confusing for code readers.

```
State.CreditAccount storage accounts = credit.accounts[msg.sender];
```

RECOMMENDATION

Consider renaming the variable to `account`.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/179c7d8b07deeec24a9e44a4c6e1ea51a411c9ab/src/components/BCredit.sol#L259>

INFORMATIONAL **I-11**

ACKNOWLEDGED

K can overflow

CATEGORY	DoS
LOCATION	Global
REVIEW	Remediation Review 1

DESCRIPTION

Currently, `K` is stored as `uint128`. This may not be enough for pools with larger supply, which will cause overflow issues.

RECOMMENDATION

Consider whether switching to `uint256` is favorable.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/260c036b75ceca66d36918dd108141963b451a0b>

CRITICAL **C-01**

ACKNOWLEDGED

Permanent BLV breakage

CATEGORY Logical Error
LOCATION .
REVIEW Remediation Review 2

DESCRIPTION

computeNextBLV() have a dangerous denominator that is built with two nested floor normalizations

```
uint256 penaltyDenom = fullMulDiv(  
    fullMulDiv(_params.supply, _params.supply, 1e18),  
    fullMulDiv(prevCirc, prevCirc, 1e18),  
    1e18  
);
```

That means penaltyDenom can become 0 even when both `params.supply > 0` and `prevCirc > 0`, once that happens, the later `fullMulDivUp` reverts

```
fullMulDivUp( , penaltyNumer, penaltyDenom)
```

The condition is

```
currentSupply * previousCirculation < 1e27 in WAD units,
```

which for 18 dec tokens is about

```
current inventory * previous circulation < 1e-9 token2
```

Example on a 18-dec pool

```
totalSupply = 10_000e18  
block-start prevCirc = 0.00001e18 tokens  
block-end supply = 0.00002e18 tokens
```

Then inside `computeNextBLV()`

```
FullMulDiv(supply, supply, 1e18) = 4e8  
FullMulDiv(prevCirc, prevCirc, 1e18) = 1e8  
outer FullMulDiv(4e8, 1e8, 1e18) = 0
```

So the next commit hits a zero denominator and reverts, allowing this

C-01 Permanent BLV breakage

CONTINUED

The poisoning swap in a block succeeds, In the next block, the first swap tries to preview/commit deferred maker state

previewDeferredMakerState() calls computeNextBLV() in the convexityExp = 2e18 path below the 95% threshold

BSwap buy / sell, BHook.beforeSwap, BCredit.leverage, deleverage are dosed

Because every future interaction retries the same poisoned commit first, the pool is stuck until privileged intervention.

There is no public recovery path that clears blockPricing or overwrites maker state

So the pool stays bricked across later blocks until doing a contract upgrade

<https://gist.github.com/GuardianAudits/ca56bc481412b7389e46bb27169d82fa>

— RECOMMENDATION

```
uint256 penaltyDenom = FixedPointMathLib.fullMulDiv(
FixedPointMathLib.fullMulDiv(_params.supply, _params.supply, 1e18),
FixedPointMathLib.fullMulDiv(prevCirc, prevCirc, 1e18),
1e18
);
if (penaltyDenom == 0) return _params.BLV;
```

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/ca56bc481412b7389e46bb27169d82fa>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/cbf1b44c312fe610a26bd65c4369e751703c1241/src/libraries/CurveLib.sol#L314-L320>

CRITICAL C-02

ACKNOWLEDGED

Pending commit can be permanently poisoned

CATEGORY	Logical Error
LOCATION	https://gist.github.com/GuardianAudits/7e851cfc8dc8b861580c488e87ca3ccbf
REVIEW	Remediation Review 2

DESCRIPTION

Pending block commit can be permanently poisoned by an unbounded powWad, the path is

```
swapTokens() -> _advanceBlockPricing() -> _commitDeferredMakerState() ->  
previewDeferredMakerState() -> _previewTranslateConvexity() ->  
previewRecomputeMaxReserves()
```

Inside _previewRecomputeMaxReserves()

```
uint256 positionRatio = _next.maxCirc.mulWad(_params.supply)  
.divWad(_params.circ.mulWad(minSupply));  
uint256 impliedBuffer = _buffer.mulWad(  
int256(positionRatio).powWad(int256(_nNew)).toUint256()  
);
```

There is no overflow check before powWad(positionRatio, nNew)

This executes only on a below ATH buy block when convexityExp > 2e18 and the pool is still below the 95% inventory threshold. so when

the pool previously reached a large maxCirc,

later sells pushed current circ back down,

then someone does a small buy while still below the old ATH

The key is that the protocol swap guards only bound the exponent for the current trade ratio supply/circ or circ/supply inside computeSwap

But _previewRecomputeMaxReserves() uses a different base

```
positionRatio = (maxCirc supply) / (circ (totalSupply - maxCirc))
```

That base can be much larger than any ratio checked during swaps

A reachable shape is

C-02 Pending commit can be permanently poisoned

CONTINUED

```
previous ATH: maxCirc = 0.99 * totalSupply
current state after sells: circ = 0.06 * totalSupply
current convexityExp = 20e18
```

Then current swap state ratio is only about $0.94 / 0.06 = 15.7$, which is well inside the normal computeSwap safety bound, but

```
positionRatio = (0.99 * 0.94) / (0.06 * 0.01) = 1551
```

For a tiny below ATH buy, `_previewTranslateConvexity()` computes `nFloor` from price continuity, and as the buy size goes to zero, `nFloor` approaches the current `convexityExp`. So `nFloor` can stay around 19–20e18

At that point

```
ln(1551) * 19 > 135
```

So `powWad(positionRatio, nFloor)` overflow / revert

The triggering buy succeeds in its own block, because the maker update is deferred

Then on the next block, every future swap starts with `_advanceBlockPricing()`

`advanceBlockPricing()` must commit the old pending block,

The commit reenters the same reverting `_previewRecomputeMaxReserves()` path, so the pending block is never cleared

That means the pool gets stuck with a poisoned `State.blockPricing` entry. After that all of these revert for that pool, Swaps, Hook swaps, Leverage, deleverage

The pool stays bricked across later blocks until doing a contract upgrade

<https://gist.github.com/GuardianAudits/7e851cfc8dc8b861580c488e87ca3ccbf>

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/e0b059fe396b5f3ecf3e329b8d5a674f>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/cbf1b44c312fe610a26bd65c4369e751703c1241/src/libraries/MakerLib.sol#L356-L385>

HIGH **H-01**

ACKNOWLEDGED

Active price mismatch censors exact buys

CATEGORY DoS
LOCATION .
REVIEW Remediation Review 2

DESCRIPTION

quoteBuyExactOut() mixes two different states in the same calculation

amountIn comes from quoteSwap(), which uses _previewBlockPricing() and prices off the frozen block start snapshot

but initialPrice comes from computeActivePrice(getCurveParams()), which can reflect the live pool state in the current block

on buys below BUFFER_SAFETY_THRESHOLD, _updateAccounting() immediately keeps part of the fee inside pool.totalReserves, raising the live active price

that means after one buy, the live initialPrice can be higher than the frozen marginal price used for the quote

when that happens, slippage = (effectivePrice - initialPrice) / initialPrice in quoteBuyExactOut() underflows and reverts

```
uint256 initialPrice = CurveLib.computeActivePrice(p);  
uint256 effectivePrice = amountIn / amountOut;  
slippage_ = (effectivePrice - initialPrice).divWad(initialPrice);
```

This also breaks exact input buys, because _solveBuy() depends on _quoteBuyCost(), which calls quoteBuyExactOut()

So buyTokensExactIn() can be DoSed

RECOMMENDATION

Compute InitialPrice from the frozen snapshot

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/a4a0fba523dd5f28a74a0c1447b21773>

H-01 Active price mismatch censors exact buys

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Stale invariant doesn't account for fees

CATEGORY Logical Error
LOCATION MakerLib.sol#L295-299
REVIEW Remediation Review 2

DESCRIPTION

During each swap, the swap fee is added to the pool's liquidity only if the circulating supply is high enough

```
if (uint256(pool.totalBTokens) < uint256(pool.totalSupply).mulWad(BUFFER_SAFETY_THRESHOLD)) {  
    liquidityFee_ = _feesReceived - 'FEE_DISTRIBUTION_SHARE.mulWad(_feesReceived);  
}
```

Even though the curve parameters are frozen for the duration of the block, `totalPoolTokens` is updated after every swap. At the start of the next block, the `invariant` will be refreshed only if the new curve's circulating is not above the `BUFFER_SAFETY_THRESHOLD`. Previously, this logic was executed after every swap, which ensured there would be no fees distributed and therefore the invariant won't change. However, with the new design there may be fees generated in the block before the curve ended up in the `BUFFER_SAFETY` region. The fees will not be accounted in the pricing mechanism because the invariant will remain stale.

This results in users getting better execution prices, and in some cases the calculation may result in user receiving both `bToken` and `reserve` during a swap. This will be caught by the validation in `MakerLib()` and users will have their transactions reverted with `InvalidSwapDirection()`

RECOMMENDATION

Consider updating the invariant if a new valid value can be computed. Keep in mind this can result in the curve changing shape even in the buffer threshold area.

<https://gist.github.com/GuardianAudits/81cd73fa047b1fc4476a087ee75f38c6>

PROOF OF CONCEPT https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/stale_invariant_dos/test/unit/pocs/StaleInvariant.t.sol

M-01 Stale invariant doesn't account for fees

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/cbf1b44c312fe610a26bd65c4369e751703c1241/src/libraries/MakerLib.sol#L269>

MEDIUM **M-02**

ACKNOWLEDGED

Mixed direction block pricing not correct

CATEGORY Logical Error
LOCATION BlockPricingLib.sol
REVIEW Remediation Review 2

DESCRIPTION

BlockPricingLib says swaps should be charged on a frozen block start curve as

```
Q(cumulativeAfter) - Q(cumulativeBefore)
```

But the implementation does not track one cumulative signed supply delta

It tracks two separate one sided deltas, blockBuyDeltaCirc and blockSellDeltaCirc

and cumulativeQuoteBounds() uses only the accumulator that matches the current trade direction

That means a buy after earlier sells in the same block is priced as if those sells never happened

as well a sell after earlier buys in the same block is priced as if those buys never happened

even though _updateAccounting() is mutating pool.totalBTokens and pool.totalReserves after every swap

So _cumulativeQuoteBounds() does this logically

For a buy, price from `buyCum -> buyCum + delta`

For a sell, price from `-sellCum -> -(sellCum + delta)`

a buy only sees earlier buys, and a sell only sees earlier sells

In a frozen curve cumulative pricing, the next trade must use the net signed block flow

RECOMMENDATION

TBD

M-02 Mixed direction block pricing not correct

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Swaps revert due to fee subtraction underflow

CATEGORY DoS
LOCATION BlockPricingLib.sol#L148-166
REVIEW Remediation Review 2

DESCRIPTION

`BlockPricingLib._quoteDeltaFromSnapshot()` is computing the fee to be paid by the user as the delta between `deltaUserWad` and `deltaInvariantWad` depending on the trade direction

The two delta variables are computed as it follows:

```
int256 deltaUserWad = afterUserWad - beforeUserWad;  
int256 deltaInvariantWad = afterInvWad - beforeInvWad;
```

Let's say $f(x) = \text{fee of } Q(x)$. Due to buffer roundings and `zeroFloorSub`s, a possible state is $f(\text{cumulativeAfter}) \leq f(\text{cumulativeBefore})$. Then the relationship between the two delta variables is: `deltaUserWad > deltaInvariantWad`

When the code reaches the `if/else` block it will try to compute `feesReceived_`, but the subtraction will result in an underflow and the swap will fail

```
if (deltaUserWad < 0) {  
    // BUY: ceil user payment, ceil curve need, fee is residual  
    uint256 userPayNative = NormalizeLib.denormalizeWadUp(uint256(-deltaUserWad), rDec);  
    uint256 curveNeedNative = NormalizeLib.denormalizeWadUp(uint256(-deltaInvariantWad), rDec);  
    deltaUserReserves_ = -(userPayNative.toInt256());  
    feesReceived_ = userPayNative - curveNeedNative;  
} else {  
    // SELL: floor user receipt, floor curve release, fee is residual  
    uint256 userReceiveNative = NormalizeLib.denormalizeWad(uint256(deltaUserWad), rDec);  
    uint256 curveReleaseNative = NormalizeLib.denormalizeWad(uint256(deltaInvariantWad), rDec);  
    deltaUserReserves_ = int256(userReceiveNative);  
    feesReceived_ = curveReleaseNative - userReceiveNative;  
}
```

M-03 Swaps revert due to fee subtraction underflow

CONTINUED

— RECOMMENDATION

If this behavior is not acceptable, you can cap the fee at 0

<https://gist.github.com/GuardianAudits/14f0e340c04a97546e3b92efc044953e>

PROOF OF CONCEPT https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/buffer_rounding_sell_dos_poc/test/unit/pocs/BufferRoundingSellDOS.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/e9d290ecd4f9e7b93c3c9a1b2cc31d522161aec0/src/libraries/BlockPricingLib.sol#L157>

MEDIUM **M-04**

ACKNOWLEDGED

Lower domain failure in `_tryRecomputeMaxReserves`

CATEGORY DoS
LOCATION CurveLib.sol
REVIEW Remediation Review 2

DESCRIPTION

we only defends the positionRatio > 1 overflow side

```
if (  
    positionRatio > WAD &&  
    _nNew.mulWad(uint256(int256(positionRatio).lnWad())) > 135e18  
) {  
    return false;  
}
```

but the function does not defend the positionRatio = 0 or tiny positive < 1 side before calling

```
uint256 growth = int256(positionRatio).powWad(int256(_nNew)).toUint256();
```

That matters because positionRatio is built with two floor divisions

```
num = fullMulDiv(_next.maxCirc, _params.supply, WAD);  
den = fullMulDiv(_params.circ, minSupply, WAD);  
positionRatio = fullMulDiv(num, WAD, den);
```

So in below ATH high convexity pools, positionRatio can floor all the way to 0 when historical maxCirc is tiny relative to current circ

Example Inside `_tryRecomputeMaxReserves`

```
num = floor(1 * 9400e18 / 1e18) = 9400  
den = floor(600e18 * (10000e18 - 1) / 1e18) ≈ 6_000_000e18  
positionRatio = floor(9400 * 1e18 / 6_000_000e18) = 0
```

So the next block deferred commit hits `powWad(0, _nNew)` and reverts

The impact is the same poisoned deferred state pattern but with lower likelihood

M-04 Lower domain failure in _tryRecomputeMaxReserves

CONTINUED

RECOMMENDATION

Keep _tryRecomputeMaxReserves fail closed on the lower side too

```
uint256 positionRatio = FixedPointMathLib.fullMulDiv(num, WAD, den);
if (positionRatio == 0) return false;
if (positionRatio > uint256(type(int256).max)) return false;
if (
    positionRatio > WAD &&
    _nNew.mulWad(uint256(int256(positionRatio).lnWad())) > 135e18
) {
    return false;
}
uint256 growth = int256(positionRatio).powWad(int256(_nNew)).toUint256();
if (growth == 0) return false;
```

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/6746e481bb97ad4617fbca7aa7638121713aa7ad/src/libraries/MakerLib.sol#L447>

MEDIUM **M-05**

ACKNOWLEDGED

Sell sign inversion from invariant rounding

CATEGORY DoS
LOCATION CurveLib.sol
REVIEW Remediation Review 2

DESCRIPTION

when the pool is in the supply < circ regime, the protocol compresses the buffer into lastInvariant with one rounding direction, then later expands it back with the opposite rounding direction

computeInvariant() stores K using

```
invRatio = circ.divWad(supply) (round down)
K = buffer.divWadUp(powWad(invRatio, n))
```

computeSwap() later reconstructs the post trade buffer using

```
invRatio = c1.divWadUp(x1) (round up)
newBuffer = fullMulDivUp(K, powWad(invRatio, n), WAD)
```

That floor to ceil asymmetry is not an inverse map. In high convexity / low buffer supply < circ states, a sell can reconstruct a newBuffer that is too large

once that happens, the sell path can do something impossible

priceAfter >= priceBefore on a sell

and in worse cases newReserves > oldReserves, so invariantDelta becomes negative on a sell

That bad sign then blows up the sell fee path, because _computeFee() assumes sell side

_invariantDelta > 0 and does uint256(_invariantDelta)

State example In the poc

```
totalSupply = 1_000_000e18
totalBTokens (supply) = 200_000e18
circ = 800_000e18
BLV = 0.1e18
totalReserves = BLV * circ + 200e18 = 80_200e18
convexityExp = 30e18
lastInvariant = 174
```

M-05 Sell sign inversion from invariant rounding

CONTINUED

Now if we sell 1e18 bToken

```
priceBefore = 0.1375e18
priceAfter = 0.1376068710996149e18 <- price rises on a sell
invariantDelta = -0.47073133411459367e18 <- negative on a sell
```

In that same state, sells of 0.1, 0.5, 1, and 2 tokens all hit the negative invariantDelta region

The first positive output sell only appears around 5 tokens, and even there the sell still raises price

_computeFee() sell branch casts uint256(_invariantDelta) on a negative int that wraps to a huge uint then fee_toInt256() reverts

So once a pool enters this region, legitimate sell side flows could become uncancellable

<https://gist.github.com/GuardianAudits/9156c953cdbc2d0a553b71dbdeb631a5>

— RECOMMENDATION

In the computeSwap() $x1 < c1$ branch

```
uint256 invRatio = c1.divWad(x1); // not divWadUp
```

This matches the supply < circ rounding used when computeInvariant() stores K, so sells stop overstating newBuffer

But do it for sell only, because the $x1 < c1$ branch is used by buys too

```
uint256 invRatio = _deltaCirc < 0 ? c1.divWad(x1) : c1.divWadUp(x1);
```

This might not completely mitigate it, but it will make the state harder to reach,

Also consider acknowledging if the Impact is acceptable

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/b8f2f676f0aec5afe43ff6328b9ca80b89ab856f#diff-8e44c461e66260304fd520823bda1095beba7d19befbf59e73f62d5d852d6bc>

LOW **L-01**

ACKNOWLEDGED

Dripping asymmetry when pending yield is 0

CATEGORY Gaming
LOCATION BStaking.sol#L231-234
REVIEW Remediation Review 2

DESCRIPTION

BStaking._sync() refreshes tokensPerSecond and lastUpdated when there is no accumulator change because of 0 pendingYield

```
else if (pool.pendingYield == 0 || staking.totalStaked == 0) {  
    staking.tokensPerSecond = tokensPerSecond.toUint128();  
    staking.lastUpdated = block.timestamp.toUint32();  
}
```

Once new yield is generated, the instantly distributable amount will depend on whether _sync() was called in the 0 yield period. If no interaction triggers _sync() when yield is 0, tokensPerSecond will still be stuck, allowing a big part of the yield (if not the whole) to be distributed as soon as it enters the system.

RECOMMENDATION

You can execute a 0 deposit in FeeLib.distributeFees() if the yield was previously 0. This will trigger a _sync() right before the new yield arrives and tokensPerSecond will be properly updated.

```
if (remaining > 0){  
    if (_pool.pendingYield == 0) {  
        BStaking(address(this)).deposit(_bToken, address(this), 0);  
    }  
    _pool.pendingYield += remaining;  
}
```

PROOF OF CONCEPT https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/staking_drip_poc/test/pocs/StakingDripCollapsePoC.t.sol

RESOLUTION

Status: Acknowledged.

L-01 Dripping assymetry when pending yield is 0

CONTINUED

<https://github.com/0xBaseline/mercury/commit/be9e2799f3e02f89ff1d0e3e2adf084130664dd4>

LOW **L-02**

ACKNOWLEDGED

BLens may return stale curve parameters

CATEGORY	Logical Error
LOCATION	BLens.sol
REVIEW	Remediation Review 2

— DESCRIPTION

The `BLens` functions `blvPrice()`, `convexityExponent()` and `lastInvariant` read the parameters' value from the committed state. This will result in integrators getting stale data when there is a pending state to be committed.

— RECOMMENDATION

Consider calling `MakerLib.getCurveParams()` and reading each value from it.

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/cbf1b44c312fe610a26bd65c4369e751703c1241/src/components/BLens.sol#L315-L330>

INFORMATIONAL **I-01**

ACKNOWLEDGED

BCredit.claim() integration notes

CATEGORY	Documentation
LOCATION	BCredit.sol#L122
REVIEW	Remediation Review 2

DESCRIPTION

`BCredit.claim()` allows anyone to initiate the claim for a given user, providing `asNative`. This can result in locked funds if integrators are not aware of it. For example, if they are using a contract that operates with only one type of token, anyone can trigger a claim with the inverse `asNative` value to lock the funds.

RECOMMENDATION

Document this, so integrators can handle both types of rewards.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/602b4a6a9e1db9327356b5c9652f16f93032cbdc/src/components/BStaking.sol#L119>

INFORMATIONAL I-02

ACKNOWLEDGED

Not all tokens can be bought

CATEGORY	Informational
LOCATION	CurveLib.sol#L111
REVIEW	Remediation Review 2

DESCRIPTION

If a swap tries to buy all the tokens from the pool, `x1` will be calculated as 0.

```
uint256 x1 = (_params.supply.toInt256() - _deltaCirc).toUint256();
```

Then it will be used as a denominator to compute `invRatio`, which will result in a revert and inability to buy all tokens from the pool.

```
uint256 invRatio = c1.divWadUp(x1);
```

Even if a trade leaves 1 wei sitting in the pool, the resulting value of `invRatio` will be huge and the transaction will revert when performing next calculations. This creates a limit of how much tokens can be bought.

RECOMMENDATION

Be aware of this behavior of the pool.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Reallocated fees commit divergency

CATEGORY DoS
LOCATION MakerLib.sol#L272-293
REVIEW Remediation Review 3

DESCRIPTION

When the pool is in the safety region, `_commitDeferredMakerState()` computes the expected reserves derived from the implied buffer. Then it compares this value with the stored `totalReserves`, and the excess is distributed as fees and subtracted from `totalReserves`. There are a few problems with this approach:

The fee removal logic is not replayed in `_previewDeferredMakerState()`. This leads to all quoting functions reading an inflated value for `totalReserves`. When a quote calls `CurveLib.computeSwap`, the `invariantDelta` value will also be inflated.

```
invariantDelta_ = _params.reserves.toInt256() - newReserves;
```

This can result in a positive `invariantDelta_` for buys, DOS-ing the `BSwap.buyTokensExactIn()` flow. For sells, the inflated `invariantDelta_` value will result in wrong fee being calculated when the `zeroFloorSub` case is not entered.

Because `totalReserves` is artificially increased until the fee is settled, `GuardLib.ensureSolvent()` becomes less restrictive and the risk of the protocol being left in a non-solvent state increases.

```
function ensureSolvent(BToken _bToken) internal view {  
    require(State.pool(_bToken).totalReserves >= State.credit(_bToken).totalDebt,  
        GuardLib_Insolvent());  
}
```

This discrepancy will also affect third-party integrators that read from `BLens`.

The way the fee surplus is calculated based on the implied buffer and implied reserves is also risky because these calculations are never exact.

The current design of taking fees out of the pool opens up a new attack vector where stakers can push the pool in the safety regime intentionally in order to get these fees distributed to them. It becomes easier for block builders because they can order the transactions in a way that benefits them and requires minimal funds from them.

M-01 Reallocated fees commit divergency

CONTINUED

— RECOMMENDATION

If the staker risk is acceptable and you plan to stick to the current design, consider tracking potential fees in a separate variable, not in `totalReserves`. Then correct the preview logic to account for that variable as well.

PROOF OF CONCEPT https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/commit_divergency_poc/test/unit/pocs/CommitPreviewDivergence.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/pull/172>

LOW **L-01**

ACKNOWLEDGED

Undistributed fees when circ reaches zero

CATEGORY Logical Error
LOCATION MakerLib.sol
REVIEW Remediation Review 3

DESCRIPTION

When `_updateAccounting()` charges a liquidity fee, it retains a portion of swap fees in `totalReserves`. The surplus-distribution branch in `_commitDeferredMakerState()` is responsible for distributing these retained fees by reverse-engineering the surplus from the curve invariant. However, this branch is guarded by `params.circ > 0` because the implied-reserves calculation requires `supply.divWad(circ)`, which would revert on division by zero.

If multiple sells occur in the same block where earlier sells generate liquidity fees (pool below `BUFFER_SAFETY_THRESHOLD`) and a later sell pushes `totalBTokens == totalSupply` (circulating supply to zero), the liquidity fees from the earlier sells remain in `totalReserves` but the surplus-distribution branch is skipped due to `circ == 0`. The curve adjustment branch in `_previewDeferredMakerState()` also does not fire since `supply == totalSupply` is well above the 95% threshold.

The retained liquidity fees are never distributed to stakers, protocol, or creator via `distributeFees()`. When trading resumes and a buy makes `circ` non-zero again, the next commit refreshes `K` from the current state, absorbing the undistributed fees into the new invariant permanently.

Additionally, the surplus recomputation is inherently imprecise. `lastInvariant` is itself a rounded value, and the recovery path applies further rounding at each step — `divWad()` on the supply/circ ratio, `powWad()` for the exponentiation, `fullMulDivUp()` for the implied buffer, and `mulWadUp()` for the BLV component. Each layer compounds the inaccuracy, meaning the recomputed surplus can understate the actual retained fee even when `circ` is non-zero.

RECOMMENDATION

Track retained liquidity fees in a dedicated accumulator (e.g. `pool.pendingSurplus`) at the time they are charged in `_updateAccounting()`. In `_commitDeferredMakerState()`, distribute the accumulated value directly via `distributeFees()` and reset the accumulator. This eliminates the dependency on curve math for fee recovery and avoids both the `circ == 0` edge case and the cumulative rounding inaccuracy.

L-01 Undistributed fees when circ reaches zero

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/pull/172>

LOW **L-02**

ACKNOWLEDGED

Buffer threshold precision mismatch

CATEGORY Math
LOCATION MakerLib.sol
REVIEW Remediation Review 3

DESCRIPTION

The `BUFFER_SAFETY_THRESHOLD` check uses native `bToken` precision in `_updateAccounting()` and `_commitDeferredMakerState()`, but wad (18-decimal) precision in `_previewDeferredMakerState()`.

`_updateAccounting()` and `_commitDeferredMakerState()` compare raw `pool.totalBTokens` against `pool.totalSupply.mulWad(BUFFER_SAFETY_THRESHOLD)`. `_previewDeferredMakerState()` compares `currentParams.supply` against `currentParams.totalSupply.mulWad(BUFFER_SAFETY_THRESHOLD)`, where both operands were normalized to 18 decimals via `NormalizeLib.normalizeWad()`.

For `bTokens` with fewer than 18 decimals, `mulWad()` truncation at native precision produces a strictly lower threshold than the wad-precision check. At the boundary where `totalBTokens == floor(totalSupply * 0.95e18 / 1e18)`, the native `>=` check in `_commitDeferredMakerState()` evaluates to true and the wad `<` check in `_previewDeferredMakerState()` also evaluates to true. Both branches in `_commitDeferredMakerState()` execute despite being mutually exclusive.

When both branches fire, the curve-adjustment branch in `_previewDeferredMakerState()` runs first, recomputing the invariant `K` using `currentParams.reserves` which includes any liquidity fees added to `totalReserves` by `_updateAccounting()`. The new `K` absorbs these fees. Then the surplus-distribution branch in `_commitDeferredMakerState()` reads the freshly stored `K`, computes `impliedReserves`, and finds no surplus because `K` already accounts for the fee.

The result is that liquidity fees collected during swaps below the threshold get permanently locked into the curve invariant rather than being distributed to stakers, protocol, and creator via `distributeFees()`. In correct behavior (safety region, `K` frozen), the old `K` would not account for the fee, yielding a distributable surplus.

This affects pools with `bTokenDecimals < 18` when `totalBTokens` lands exactly at the native threshold boundary.

L-02 Buffer threshold precision mismatch

CONTINUED

— RECOMMENDATION

Use the same precision for all three threshold checks. Make `_previewDeferredMakerState()` use native precision, matching `_updateAccounting()` and `_commitDeferredMakerState()`. Read the pool state directly instead of relying on the wad-normalized `currentParams`:

```
State.Pool storage pool = State.pool(_bToken);  
if (uint256(pool.totalBTokens) < uint256(pool.totalSupply).mulWad(BUFFER_SAFETY_THRESHOLD)) {
```

PROOF OF CONCEPT https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/buffer_threshold_mismatch_poc/test/unit/pocs/BufferThresholdMismatch.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/995abfcc4f767feaad059e703ee4867c92018375>

LOW **L-03**

ACKNOWLEDGED

Fee rounding favors buyer over protocol

CATEGORY Rounding
LOCATION CurveLib.sol
REVIEW Remediation Review 3

DESCRIPTION

In `_computeFee()`, `marginalPremium` is computed using floor rounding throughout — `fullMulDiv()` for the outer division, and `mulWad()` for both the numerator (`convexityExp * totalSupply`) and denominator (`c1 * x1`) — regardless of trade direction:

```
uint256 marginalPremium = FixedPointMathLib.fullMulDiv(
    _newBuffer,
    _p.convexityExp.mulWad(_p.totalSupply),
    c1.mulWad(x1)
);
```

For sells this is partially correct — floor rounding on the numerator `mulWad()` and outer `fullMulDiv()` understates `marginalReceipt`, overstates the fee, and the protocol captures the dust upfront. However, the denominator `c1.mulWad(x1)` also floors, which makes the fraction larger and partially counteracts the understatement — ideally it should use `mulWadUp()` for sells to consistently round down.

For buys, the uniform floor rounding understates `marginalCost`, which understates the fee. Since `userDelta_ = invariantDelta_ - fee_`, the understated fee makes `userDelta_` less negative — the buyer pays less while fee recipients receive less.

The comment claims “~1 WAD wei dust stays in pool reserves rather than being extracted as fee,” but this is inaccurate:

1. For buys, the dust leaks to the user, not to the pool.
2. The pool's reserves are determined by `invariantDelta_` (fixed by ceiled `newBuffer`), so they are unaffected by fee rounding in either direction.

Additionally, the block pricing mechanism in `BlockPricingLib` differences two cumulative `computeSwap()` calls. If the first buy's cumulative endpoint rounds down, the second buy's baseline shifts, causing it to overpay. With floor rounding, if no subsequent swap occurs, the dust from the understated fee is simply never collected. With ceil rounding, the protocol would have already captured it from the first buyer.

L-03 Fee rounding favors buyer over protocol

CONTINUED

RECOMMENDATION

Round `marginalPremium` up for buys and down for sells, applying consistent rounding to all intermediate operations:

```
uint256 marginalPremium;
if (_deltaCirc > 0) {
    // BUY: round up to not understate fee
    marginalPremium = FixedPointMathLib.fullMulDivUp(
        _newBuffer,
        _p.convexityExp.mulWadUp(_p.totalSupply),
        c1.mulWad(x1)
    );
} else {
    // SELL: round down to not overstate receipt
    marginalPremium = FixedPointMathLib.fullMulDiv(
        _newBuffer,
        _p.convexityExp.mulWad(_p.totalSupply),
        c1.mulWadUp(x1)
    );
}
```

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/master/src/libraries/CurveLib.sol#L165-L184>

LOW **L-04**

ACKNOWLEDGED

Stale BLV in BCredit and quoteLeverage

CATEGORY	Logical Error
LOCATION	BCredit.sol BLens.sol
REVIEW	Remediation Review 3

DESCRIPTION

`BCredit` and `BLens.quoteLeverage()` read `blvPrice` directly from committed storage via `State.maker(_bToken).blvPrice` instead of previewing the deferred state through `MakerLib.getCurveParams()`. The committed `blvPrice` is only updated when `_commitDeferredMakerState()` runs, which is triggered by swaps via `_advanceBlockPricing()`. In a new block with uncommitted state from the previous block's trades, any call to these functions before a swap sees a stale BLV.

In `BCredit`, three functions are affected: `getBorrowForCollateral()`, `getMaxBorrow()`, and `_previewBorrow()`. All compute max debt capacity as `blv.mulWad(collateralWad)`. Since BLV is monotonically increasing, the stale value is always lower, meaning users receive less borrowing power than they should.

`BLens.quoteLeverage()` has an additional inconsistency: it calls `activePrice(_bToken)` which correctly previews the deferred state through `MakerLib.getCurveParams()`, but then subtracts the stale `maker.blvPrice` from it. The expression `activePrice(_bToken) - maker.blvPrice` mixes a live value with a stale one. On top of that, it calls `BCredit.getBorrowForCollateral()` which itself uses the stale BLV, so `borrowAmount` is already undervalued before the leverage math begins.

The inaccuracy could cause failed transactions when a user acts on a stale quote that underestimates their capacity.

RECOMMENDATION

Read BLV through `MakerLib.getCurveParams()` which previews the deferred state. In `BCredit`, replace `State.maker(_bToken).blvPrice` with `MakerLib.getCurveParams(_bToken).BLV` in `getBorrowForCollateral()`, `getMaxBorrow()`, and `_previewBorrow()`. In `BLens.quoteLeverage()`, replace the direct `State.maker(_bToken)` read with the curve params from `getCurveParams()` to keep BLV consistent with the already-live `activePrice()`.

L-04 Stale BLV in BCredit and quoteLeverage

CONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

computeNextBLV() overflow DOSes pool

CATEGORY	DoS
LOCATION	CurveLib.sol#L315
REVIEW	Remediation Review 3

DESCRIPTION

In `computeNextBLV()`, the `penaltyDenom` calculation squares an intermediate value:

```
uint256 penaltyDenom = FixedPointMathLib.fullMulDiv(
    FixedPointMathLib.fullMulDiv(_params.supply, prevCirc, 1e18),
    FixedPointMathLib.fullMulDiv(_params.supply, prevCirc, 1e18),
    1e18
);
```

This computes $(\text{supply} * \text{prevCirc} / 1e18)^2 / 1e18$. When both `supply` and `prevCirc` are large enough (token supplies exceeding $\sim 1e33$ at 18 decimals), the result overflows `uint256` and `fullMulDiv()` reverts with `FullMulDivFailed()`.

Since `computeNextBLV()` is called during `_commitDeferredMakerState()`, which runs inside `_advanceBlockPricing()` at the start of every swap in a new block, the revert permanently bricks the pool — pending state from the previous block can never be committed, and all subsequent swaps revert.

RECOMMENDATION

If the use case allows, cap the accepted token `totalSupply` to `type(uint108).max` in `setBTokenDeployment()` or `createPool()`. With `uint108`-bounded values, `penaltyDenom` stays within 2^{252} , safely below `type(uint256).max`.

Additionally, `penaltyNumer` is currently rounded down via `fullMulDiv()`, which makes the `penalty` smaller than the exact value. Since `maxBLV = bookPrice - penalty`, a smaller penalty allows a higher `maxBLV`, letting BLV rise slightly past the K-preserving threshold. For correctness, `penaltyNumer` should be rounded up (using `fullMulDivUp()`) to maximize the penalty and keep `maxBLV` conservative. Note that rounding up the intermediate calculations increases the overflow risk at the boundary, which is another reason to reduce the accepted supply range from `uint128` to `uint108`.

L-05 computeNextBLV() overflow DOSes pool

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/52469512105af39cf3d1ab810b01c8ced1bacbc5/src/components/BFactory.sol#L253>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Integration note completeness

CATEGORY Documentation
LOCATION BStaking.sol#L120-121
REVIEW Remediation Review 3

DESCRIPTION

The following note was added to `BStaking.claim()` in response to I-01

```
/// @dev Since this function is permissionless all integrating contracts need to handle  
/// native token transfers or risk losing funds.
```

However, it goes both ways. A contract that works only with native tokens can have its rewards claimed with `asNative = false`, resulting in stuck rewards.

RECOMMENDATION

Modify the note to mention both types of rewards.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/1ad34294b8590506cec9dc3cd571e3e7d74e3302/src/components/BStaking.sol#L120>

LOW **L-01**

ACKNOWLEDGED

Safety threshold precision mismatch

CATEGORY Rounding
LOCATION MakerLib.sol#L621
REVIEW Remediation Review 4

DESCRIPTION

Similar to Buffer threshold precision mismatch, [MakerLib._getCommittedCurveParams()] (<https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/a1b6c91955d965c73259e4cde250cd4d8b94b432/src/libraries/MakerLib.sol#L621>) uses WAD precision when calculating whether the pool is in safety regime in order to add the pending surplus.

```
if (
    pool.pendingSurplus > 0 &&
    params_.supply < params_.totalSupply.mulWad(BUFFER_SAFETY_THRESHOLD)
) {
    params_.reserves += NormalizeLib.normalizeWad(
        uint256(pool.pendingSurplus), pool.reserveDecimals
    );
}
```

This is different from the checks in the rest of the codebase - all of them use native token decimals. It's possible for `totalBTokens` to be treated as "in safety" by the native check, while the WAD check considers it "below safety."

This causes `_getCommittedCurveParams()` to add `pendingSurplus` to `params_.reserves`, inflating the curve params passed to `_previewDeferredMakerState()` and `getCurveParams()`. The inflated reserves view leaks into `getCurveParams()` and `getSnapshotCurveParams()`, which could produce incorrect quotes, potentially also impacting swaps through `BSwap` solver paths.

RECOMMENDATION

Use the native tokens decimal precision.

```
if (
    pool.pendingSurplus > 0 &&
    - params_.supply < params_.totalSupply.mulWad(BUFFER_SAFETY_THRESHOLD)
    + uint256(pool.totalBTokens) <
```

L-01 Safety threshold precision mismatch

CONTINUED

```
uint256(pool.totalSupply).mulWad(BUFFER_SAFETY_THRESHOLD)
){
params_.reserves += NormalizeLib.normalizeWad(
uint256(pool.pendingSurplus), pool.reserveDecimals
);
}`
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/a1b6c91955d965c73259e4cde250cd4d8b94b432/test/unit/pocs/SafetyRounding.t.sol>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/pull/172/changes/696c458286b7a7bd785825325ef476a9cd84079>

LOW **L-02**

ACKNOWLEDGED

View reserves inflated by unstripped surplus

CATEGORY Unexpected Behavior
LOCATION MakerLib.sol#L598
REVIEW Remediation Review 4

DESCRIPTION

`_getCommittedCurveParams()` returns live `totalReserves` without simulating the batching-asymmetry surplus stripping that `_commitDeferredMakerState()` performs at L301-321. When both buys and sells occur in the same block while in safety (pooled $\geq 95\%$), the independent block-batched pricing creates a reserve surplus due to curve convexity — actual `totalReserves` exceed the K-implied reserves. This surplus is stripped at commit time, but view functions that read `_getCommittedCurveParams()` see the inflated value.

Additionally, the existing `pendingSurplus` injection at L619-626 runs unconditionally for any call where the pool is below safety, including intra-block calls. This means `getCurveParams()` previews liquidity fees being rolled into reserves mid-block, even though the actual rollover only happens at commit time and the final safety state at commit may differ from the mid-block state.

Affected downstream consumers:

- `getCurveParams()` (L555) — returns inflated reserves in both same-block and cross-block paths
- `activePrice()` via `BLens` (L316) — computes price from inflated reserves
- `swapTokens()` Swap event (L148-152) — emits `activePrice` from `getCurveParams()` with inflated reserves
- `_previewBlockPricing()` cross-block path (L507-510) — uses inflated `committed.reserves` via `curveParamsFromDeferredState()`

Same-block swap pricing and quoting are not affected because `_previewBlockPricing()` uses the frozen snapshot via `applyPoolSnapshot()` (L501), which reads `startReserves` / `startSupply` rather than committed reserves.

RECOMMENDATION

L-02 View reserves inflated by unstripped surplus

CONTINUED

Limit reserve adjustments in `_getCommittedCurveParams()` to the cross-block window where committed state is fully consistent (K, reserves, supply all from the same commit point). Add batching-asymmetry surplus stripping for the in-safety case, and move `pendingSurplus` injection into the same cross-block guard:

```
function _getCommittedCurveParams(BToken _bToken) private view returns (CurveParams memory
params_) {
    State.Pool storage pool = State.pool(_bToken);
    State.Maker storage maker = State.maker(_bToken);
    uint8 bDec = pool.bTokenDecimals;
    params_.BLV = uint256(maker.blvPrice);
    params_.circ = NormalizeLib.normalizeWad(pool.totalSupply - pool.totalBTokens, bDec);
    params_.supply = NormalizeLib.normalizeWad(pool.totalBTokens, bDec);
    params_.reserves = NormalizeLib.normalizeWad(pool.totalReserves, pool.reserveDecimals);
    params_.totalSupply = NormalizeLib.normalizeWad(pool.totalSupply, bDec);
    params_.convexityExp = uint256(maker.convexityExp);
    params_.swapFee = uint256(maker.swapFee);
    params_.lastInvariant = maker.lastInvariant;
    - if (
    - pool.pendingSurplus > 0 &&
    - params_.supply < params_.totalSupply.mulWad(BUFFER_SAFETY_THRESHOLD)
    - ) {
    - params_.reserves += NormalizeLib.normalizeWad(
    - uint256(pool.pendingSurplus), pool.reserveDecimals
    - );
    - }
    + if (State.blockPricing(_bToken).blockNumber == 0
    + || State.blockPricing(_bToken).blockNumber == uint64(block.number)) {
    + return params_;
    + }
    +
    + bool inSafety = uint256(pool.totalBTokens)
    + >= uint256(pool.totalSupply).mulWad(BUFFER_SAFETY_THRESHOLD);
    +
    + if (inSafety) {
    + if (params_.circ > 0) {
    + uint256 ratio = params_.supply.divWad(params_.circ);
    + uint256 ratioPowN = int256(ratio).powWad(
    + params_.convexityExp.toInt256()).toUint256();
    + uint256 impliedBuffer = FixedPointMathLib.fullMulDivUp(
```

L-02 View reserves inflated by unstripped surplus

CONTINUED

```
+ params_.lastInvariant, WAD, ratioPowN);
+ uint256 impliedReserves = impliedBuffer
+ + params_.BLV.mulWadUp(params_.circ);
+ if (params_.reserves > impliedReserves) {
+   uint256 surplusNative = NormalizeLib.denormalizeWad(
+   params_.reserves - impliedReserves, pool.reserveDecimals);
+   if (surplusNative > 0) {
+     params_.reserves -= NormalizeLib.normalizeWad(
+     surplusNative, pool.reserveDecimals).toUint128();
+   }
+ }
+ }
+ } else if (pool.pendingSurplus > 0) {
+   params_.reserves += NormalizeLib.normalizeWad(
+   uint256(pool.pendingSurplus), pool.reserveDecimals);
+ }
}
```

This scopes adjustments to cross-block views only. Intra-block, `getCurveParams().reserves` still reflects raw `totalReserves` including any surplus accumulated so far, since stripping mid-block would mix stale K with live reserves. Actual swap pricing is unaffected (frozen snapshot), and the view self-corrects once the next block's commit runs.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/c3332e09/test/unit/pocs/BatchingSurplusPreview.t.sol>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/commit/be30c2b7c48feeb3d2ee827bed03843a614f06a2>

INFORMATIONAL I-01

ACKNOWLEDGED

Quote slippage is cumulative, not per-trade

CATEGORY Informational
LOCATION BSwap.sol#L232
REVIEW Remediation Review 4

DESCRIPTION

All four quote functions (`quoteBuyExactOut()`, `quoteBuyExactIn()`, `quoteSellExactIn()`, `quoteSellExactOut()`) compute slippage as $(\text{effectivePrice} - \text{initialPrice}) / \text{initialPrice}$, where `initialPrice` is derived from `getSnapshotCurveParams()` — the frozen block-start curve. The `effectivePrice` is the average price the caller would pay, priced on the cumulative curve that accounts for all prior same-block trades.

Because `initialPrice` is the block-start snapshot price rather than the marginal price at the caller's cumulative position, the returned slippage measures deviation from the start-of-block price, not the caller's personal price impact. For the first trade in a block both values coincide. For every subsequent trade, the reported slippage includes the price movement caused by earlier same-block trades.

RECOMMENDATION

Document that the slippage returned by quote functions represents the cumulative block slippage, not the individual caller's price impact. Integrators who need per-trade slippage should compute the marginal price at the current cumulative position and measure their effective price against that instead.

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/e821189589e6d3aa2f809df0b638c5bf3b04ab87/src/components/BSwap.sol#L186>

MEDIUM **M-01**

ACKNOWLEDGED

Missed removal leave hidden extractable reserves

CATEGORY Logical Error
LOCATION MakerLib.sol
REVIEW Remediation Review 5

DESCRIPTION

In the next block commit path for pools at or above the 95% safety threshold, `commitDeferredMakerState()` is supposed to remove any safety region surplus from `pool.totalReserves` and reclassify it as fees

```
CurveParams memory params = _getCommittedCurveParams(_bToken);  
if (params.reserves > impliedReserves) {  
    pool.totalReserves -= surplusNative;  
    pool.distributeFees(_bToken, surplusNative);  
}
```

But `getCommittedCurveParams()` is not raw storage. when block pricing is stale and the pool is in safety, it already applies the same skim in memory

```
if (params_.reserves > impliedReserves) {  
    params_.reserves -= normalizedSurplus;  
}
```

So `commitDeferredMakerState()` asks for committed params, but receives preview. skimmed params

As a result, by the time it checks

```
if (params.reserves > impliedReserves)
```

the surplus has already been removed from `params.reserves` in memory, so the storage side branch often never executes, that means

```
pool.totalReserves -= surplusNative;  
pool.distributeFees(_bToken, surplusNative);
```

does not run, and the real surplus remains stranded inside `pool.totalReserves`

Exploit flow

M-01 Missed removal leave hidden extractable reserves

CONTINUED

- 1 - A block ends with the pool still in safety and with `totalReserves > impliedReserves`
- 2 - On the next block, `advanceBlockPricing()` calls `commitDeferredMakerState()`
- 3 - The commit fails to remove the surplus because `getCommittedCurveParams()` already skimmed it in memory
- 4 - `BlockPricingLib.initialize()` uses that skimmed view, so the new pricing snapshot is based on lower reserves
- 5 - Storage still keeps the higher real `pool.totalReserves`
- 6 - The attacker buys `bTokens` against the lower skimmed snapshot, so they do not pay for the surplus
- 7 - They buy enough to move the pool below the 95% threshold
- 8 - On the following block, the below safety commit no longer applies the safety skim, so the raw higher `pool.totalReserves` is merged back into live curve state
- 9 - The attacker sells back on the now surplus backed curve and extracts the previously stranded reserves

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/8cd0ceae080304ec4c6558cdf3221aa>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Snapshot sell miscalculate circulation state

CATEGORY Logical Error
LOCATION BlockPricingLib.sol
REVIEW Remediation Review 5

DESCRIPTION

The pool takes a snapshot at the start of the block of how many tokens are circulating outside the pool

And how much reserve is in the pool then, inside the same block

If someone buys tokens, that buy is tracked in a buy bucket, If someone sells tokens, that sell is tracked in a sell bucket

when the pool later prices a sell, it only looks at the sell bucket, it ignores the earlier buy in the same block

There is a special branch in the code for the case when this is the final sell and circulation became exactly zero

```
if (c1 == 0) {  
    uint256 blvValue = _params.BLV.mulWad(_params.circ);  
    uint256 receipt = blvValue.mulWad(WAD - (_params.swapFee*2));  
    return (receipt.toInt256(), _params.reserves - receipt, _params.reserves.toInt256());  
}
```

The pool then pay the seller a final exit amount and classify almost all the remaining reserves as fees

That logic is only correct if there are truly no tokens left outside the pool

But the pool can think, a sell took circulation from 100 down to 0

when the real live state like "circulation was 100, then someone bought 5, so it became 105, and after selling 100 it is still 5" Just for example

If circulation is not zero, there are still people outside the pool holding tokens that should still have backing

So due the issue that branch can run even though other holders still exist

This could unlock a payout path that should only exist when nobody else is left

M-02 Snapshot sell miscalculate circulation state

CONTINUED

— RECOMMENDATION

<https://gist.github.com/GuardianAudits/c752944102d6de66f83020736794dab3>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Same-block terminal exit pays below BLV floor

CATEGORY Logical Error
 LOCATION CurveLib.sol#L87
 REVIEW Remediation Review 5

DESCRIPTION

The terminal-exit branch in `CurveLib.computeSwap()` returns a fixed cumulative receipt of $BLV \cdot C_0 \cdot (1 - 2 \cdot \text{swapFee})$ when cumulative sells reach `snapshot.circ`. The inline comment states "User only receives BLV value (floor price), buffer becomes protocol fee" — implying BLV is the guaranteed floor.

In `BlockPricingLib._quoteDeltaFromSnapshot()`, this terminal-exit cumulative is split between in-block sellers via differencing:

```
seller_B_receipt = BLV * C0 * (1 - 2 * swapFee) - seller_A_cumulative
```

Let S be the tokens Seller A already sold. Seller A's cumulative receipt is computed normally on the curve and equals $BLV \cdot S + \text{bufferShareCaptured}$, where $\text{bufferShareCaptured} \geq 0$ is the proportional buffer share the curve gives intermediate sellers (always > 0 in any pool with non-zero buffer).

Two failure modes depending on Seller A's cumulative:

- $\text{seller_A_cumulative} > BLV \cdot S$ (essentially any sell capturing buffer share): residual > 0 but Seller B's per-token receipt drops below BLV (violates floor invariant)
- $\text{seller_A_cumulative} > BLV \cdot C_0 \cdot (1 - 2 \cdot \text{swapFee})$ (large first sell): $\text{deltaUserWad} < 0$, the denormalize step routes through the buy branch where `denormalizeWadUp(uint256(-deltaInvariantWad), ...)` underflows $\rightarrow$ tx reverts

POC numbers (BLV=2, $C_0=100$, $B_0=300$, swapFee=0.3%, terminal-exit ceiling=198.8):

Scenario	A's cumulative	Outcome
A sells 5	$\rightarrow 38.3$	38.3 B sells 95, receives 160.5 (1.69/token, below BLV=2); shortfall ≈ 28 ether (~96% structural, ~4% fees)
A sells 50	$\rightarrow 249$	249 B's tx reverts (direction flip)

M-03 Same-block terminal exit pays below BLV floor

CONTINUED

The post-fix `_ensureSellWithinSameBlockCapacity` `cap ( maxSellDelta = C0 - S - B )` only addresses the buy-then-terminal-exit case ($B > 0$). When $B = 0$, the cap allows cumulative sells to reach C_0 exactly, so both failure modes still trigger.

Impact: any seller pushing cumulative sells to `snapshot.circ` after another in-block seller will either receive less than the documented BLV floor or have their tx reverted — violating the floor invariant, or providing a same-block griefing/DoS vector against exit traffic.

— RECOMMENDATION

In the following gist you will find a recommended code fix, which doesn't apply `swapFee` for the final sell. If keeping the fee is the desired behavior, `_applySellFloor()` can be modified to account for it.

<https://gist.github.com/GuardianAudits/c585959c08ed97bc0987f6a48d3424bb>

Example implementation: <https://github.com/GuardianOrg/mercury-team1-1768594959614/commit/da92381268010d0dc8b28b41546db00ac41d45e6>

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1768594959614/blob/47206c45/test/unit/pocs/SameBlockSellDirectionFlip.t.sol>

— RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/pull/178>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Rebalance unlocks surplus from initial credit

CATEGORY Informational
LOCATION BCredit.sol#L478
REVIEW Remediation Review 5

— DESCRIPTION

The new `_rebalanceCollateral()` helper invoked at the end of `_borrow()`, `_repay()`, `leverage()`, and `deleverage()` recomputes locked collateral as $\text{collateral} \cdot \text{debt} / (\text{maxBorrow} + \text{fee})$ and unlocks the surplus. This unifies the locking model: locked collateral always equals the minimum required to back the current debt.

A side effect that's easy to miss: when a pool is created with an over-collateralized initial credit position ($\text{initialCollateral} > \text{requiredCollateral}(\text{initialDebt})$ per merkle leaf), the surplus is no longer bonded to the position. After claiming, a user can trigger the rebalance with any state-changing call (e.g. `borrow(1 wei)`) and immediately withdraw the surplus through `BStaking.withdraw()`, while keeping the cheap debt position open.

Pre-fix, unlocking surplus required actually repaying debt — `_previewRepay` redeems collateral in proportion to repaid debt, so the surplus was implicitly bonded to the debt's lifecycle.

Concrete example: pool created with $\text{initialCollateral}=50$, $\text{initialDebt}=5$, $\text{BLV}=2$. After claim, the position is $\text{collateral}=50$, $\text{debt}=5$, $\text{locked}=50$. The minimum required to back $\text{debt}=5$ is $50 \cdot (5 / \text{maxBorrow}) \approx 2.5$. A single `borrow(1)` call triggers rebalance and unlocks ~47.5 of collateral, withdrawable in the same tx.

This may be fine if deployers create over-collateralized initial positions purely as a safety margin / UX default. But if any deployer relies on the over-collateralization as a long-term commitment or "stake bond" against the claimant, the new fix silently invalidates that assumption.

— RECOMMENDATION

Document that initial credit position collateral is no longer "locked beyond debt requirement". Deployers who want a permanent collateral lock unrelated to the debt position must enforce it through other mechanisms (e.g. an external time-lock, a separate stake position, or by sizing `initialDebt` close to the `maxBorrow` of `initialCollateral`).

I-01 Rebalance unlocks surplus from initial credit

CONTINUED

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.